HUTECH
Đại học Công nghệ Tp.HCM

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN HỌC PHẦN

Môn học: CMP180 - Lập trình mạng máy tính

Đề tài:

TRÒ CHƠI CỜ CARO ĐỐI KHÁNG TRỰC TUYẾN (CARO ONLINE)

Giảng viên hướng dẫn: **ThS. Nguyễn Văn Cẩn**

Nhóm thực hiện: **Nhóm 9**

Thành viên 1: Đỗ Võ Ngọc Thắm - MSSV: 2380602068

Thành viên 2: Nguyễn Trần Như Ngọc - MSSV: 2380601470

Thành viên 3: Phạm Trần Khánh Luân - MSSV: 2380601295

Lớp: 23DTHA4 Năm học: 2025 - 2026

LỜI CAM ĐOAN

Nhóm chúng em xin cam đoan đồ án **"Trò chơi cờ Caro đối kháng trực tuyến (Caro Online)"** là sản phẩm do chính nhóm nghiên cứu và thực hiện trong khuôn khổ học phần Lập trình mạng máy tính, dưới sự hướng dẫn của ThS. Nguyễn Văn Cẩn.

Trong quá trình thực hiện đồ án, nhóm có tham khảo tài liệu từ Oracle, MySQL và một số bài viết kỹ thuật liên quan nhằm tìm hiểu rõ hơn về Socket, lập trình đa luồng và JDBC. Các tài liệu được sử dụng đều được liệt kê trong phần Tài liệu tham khảo của báo cáo. Trên cơ sở đó, nhóm đã tự thực hiện các phần thiết kế giao thức, xây dựng server và client, phát triển các mức độ AI cũng như thiết kế cơ sở dữ liệu, đồng thời kiểm tra và điều chỉnh để các thành phần hoạt động phù hợp với nhau.

Nhóm chúng em xin chịu hoàn toàn trách nhiệm về tính trung thực và chính xác của nội dung được trình bày trong báo cáo này.

TP. Hồ Chí Minh, ngày 22 tháng 10 năm 2026

Nhóm thực hiện

Nguyễn Trần Như Ngọc

Đỗ Võ Ngọc Thắm

Phạm Trần Khánh Luân

MỤC LỤC

LỜI CAM ĐOAN 1
MỤC LỤC 2
DANH MỤC CÁC KÝ HIỆU, CÁC CHỮ VIẾT TẮT 4
DANH MỤC CÁC BẢNG 5
DANH MỤC HÌNH ẢNH 6
CHƯƠNG 1 : TỔNG QUAN ĐỀ TÀI 7
1.1. Giới thiệu đề tài 7
1.2. Nhiệm vụ đồ án 7
1.2.1. Tính cấp thiết và lý do hình thành đề tài 7
1.2.2. Ý nghĩa khoa học và thực tiễn 8
1.2.3. Mục tiêu đề tài 8
1.2.4. Đối tượng và phạm vi giới hạn 8
1.3. Cấu trúc đồ án 9
CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT 10
2.1. Mô hình Client-Server 10
2.2. TCP Socket trong Java 10
2.3. Lập trình đa luồng (Thread) trong Java 10
2.4. Lớp InetAddress 11
2.5. Truyền đối tượng qua luồng nhập/xuất mạng 11
2.6. JDBC và MySQL 12
2.7. Java Swing 12
CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM 13
3.1. Yêu cầu hệ thống 13
3.2. Phân tích và thiết kế hệ thống 13
3.2.1. Kiến trúc tổng thể 13
3.2.2. Thiết kế lớp 14
3.2.3. Thiết kế giao thức trao đổi dữ liệu 15
3.2.4. Thiết kế cơ sở dữ liệu 17
3.3. Cài đặt chương trình 20
3.3.1. Môi trường phát triển 20
3.3.2. Cấu trúc thư mục project 20

3.3.3. Một số đoạn mã nguồn tiêu biểu 21
3.3.4. Hướng dẫn biên dịch và chạy chương trình 22
3.4. Kiểm thử và kết quả 23
3.4.1. Kịch bản kiểm thử 23
3.4.2. Kết quả chạy thử thực tế 26
3.4.3. Đánh giá ưu điểm và hạn chế 30
CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ 32
TÀI LIỆU THAM KHẢO 33
PHỤ LỤC: MÃ NGUỒN CHƯƠNG TRÌNH 34

DANH MỤC CÁC KÝ HIỆU, CÁC CHỮ VIẾT TẮT

Từ viết tắt	Ý nghĩa
TCP	Transmission Control Protocol
IP	Internet Protocol
LAN	Local Area Network (mạng nội bộ)
JDBC	Java Database Connectivity
DAO	Data Access Object
GUI	Graphical User Interface (giao diện đồ họa)
AI	Artificial Intelligence (ở đây là máy chơi cờ)
SHA-256	Secure Hash Algorithm, bản băm 256 bit
IDE	Integrated Development Environment

DANH MỤC CÁC BẢNG

Bảng 3.1. Mô tả các trường dữ liệu của lớp Message. 16
Bảng 3.2. Các loại thông điệp trong giao thức 16
Bảng 3.3. Cấu trúc bảng users 18
Bảng 3.4. Cấu trúc bảng rooms 18
Bảng 3.5. Cấu trúc bảng matches 19
Bảng 3.6. Cấu trúc bảng match_moves 19
Bảng 3.7. Kịch bản kiểm thử và kết quả thực tế 26

DANH MỤC HÌNH ẢNH

Hình 3.1. Sơ đồ kiến trúc tổng thể hệ thống Caro Online. 14
Hình 3.2. Sơ đồ tuần tự một lượt đánh trong ván online.....17
Hình 3.3. Sơ đồ quan hệ giữa các bảng trong cơ sở dữ liệu. 20
Hình 3.4. Màn hình kết nối server và đăng nhập 27
Hình 3.5. Màn hình đăng ký tài khoản và sảnh chờ 27
Hình 3.6. Phòng chờ và danh sách phòng đang chờ.....28
Hình 3.7. Ván đấu online: bên thắng (trái) và bên thua (phải).....28
Hình 3.8. Giao diện chơi với máy 29
Hình 3.9. Bảng xếp hạng và lịch sử trận đấu.....29
Hình 3.10. Cửa sổ server đang chạy.....29
Hình 3.11. Các bảng của database caro_online trong phpMyAdmin.....30
Hình 3.12. Dữ liệu bảng users trong phpMyAdmin.....30

CHƯƠNG 1 : TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

Cờ Caro (hay Gomoku) là một trò chơi quen thuộc, có lẽ nhiều người đã từng chơi ít nhất một lần trên giấy, đặc biệt là trong những giờ giải lao ở trường. Luật chơi khá đơn giản: hai người lần lượt đánh dấu X và O trên bàn cờ, người đầu tiên tạo được năm quân liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo sẽ giành chiến thắng. Chính sự đơn giản này khiến nhóm lựa chọn Caro làm đề tài cho đồ án. Nhờ không phải dành quá nhiều thời gian xây dựng luật chơi phức tạp, nhóm có thể tập trung hơn vào nội dung chính của học phần Lập trình mạng máy tính, đó là cách các máy tính kết nối với nhau và trao đổi dữ liệu để cùng tham gia một ván cờ trực tuyến.

Ở giai đoạn đầu, nhóm xây dựng một phiên bản thử nghiệm với mô hình gồm một server và hai client kết nối thông qua TCP. Phiên bản này đã đáp ứng được các chức năng cơ bản như đánh cờ và xác định kết quả thắng, thua. Tuy nhiên, khi xem xét nhu cầu sử dụng thực tế, nhóm nhận thấy chương trình vẫn còn khá nhiều hạn chế. Người chơi chưa có tài khoản, chưa có cơ chế quản lý trận đấu và kết quả cũng không được lưu lại sau khi ván cờ kết thúc.

Từ những hạn chế đó, nhóm tiếp tục phát triển và bổ sung các chức năng theo từng giai đoạn. Hệ thống được mở rộng với đăng ký, đăng nhập, sảnh chờ, tạo phòng, ghép trận, đầu hàng, đề nghị hòa, lịch sử đấu và bảng xếp hạng. Bên cạnh đó, nhóm xây dựng thêm chế độ chơi với máy tính và sử dụng MySQL để lưu trữ dữ liệu. Các chức năng cốt lõi của phiên bản ban đầu như bàn cờ 15×15, kiểm tra thắng, đồng hồ đếm thời gian và trò chuyện trong trận đấu vẫn được giữ lại, làm nền tảng cho những phần mở rộng sau này.

1.2. Nhiệm vụ đồ án

1.2.1. Tính cấp thiết và lý do hình thành đề tài

Trò chơi nhiều người chơi là một ví dụ khá điển hình khi tìm hiểu về lập trình mạng. Tuy nhiên, để xây dựng một trò chơi có thể hoạt động ổn định giữa nhiều máy tính, chỉ xử lý được luật chơi là chưa đủ. Ngay cả với một ván cờ Caro chỉ có hai người tham gia, nhóm vẫn phải giải quyết nhiều vấn đề liên quan đến kết nối và đồng bộ dữ liệu. Chẳng hạn, hệ thống cần xác định bên nào có quyền thực hiện nước đi, xử lý ra sao khi người chơi mất kết nối giữa trận, hoặc ngăn chặn trường hợp client gửi nước đi không đúng lượt. Khi có nhiều trận đấu diễn ra đồng thời, server cũng phải quản lý các phiên chơi riêng biệt để tránh ảnh hưởng lẫn nhau.

Xuất phát từ những vấn đề đó, nhóm mong muốn vận dụng kiến thức đã học để tự xây dựng một ứng dụng hoàn chỉnh, thay vì chỉ dừng lại ở việc tìm hiểu lý thuyết. Cờ Caro được lựa chọn vì có luật chơi dễ tiếp cận, phạm vi vừa sức với nhóm ba thành viên trong thời gian một học kỳ, nhưng vẫn đặt ra những yêu cầu thực tế về kết nối mạng, xử lý đồng thời và quản lý dữ liệu. Qua quá trình thực hiện, nhóm có thể hiểu rõ hơn những khó khăn thường gặp khi phát triển một ứng dụng có nhiều người dùng cùng tham gia.

1.2.2. Ý nghĩa khoa học và thực tiễn

Về mặt học thuật, đồ án là cơ hội để nhóm vận dụng nhiều kiến thức của học phần Lập trình mạng máy tính vào cùng một sản phẩm. Các nội dung được sử dụng bao gồm mô hình Client-Server, Socket TCP, lập trình đa luồng, truyền đối tượng qua luồng mạng và kết nối cơ sở dữ liệu bằng JDBC. Mặc dù những kiến thức này đã được tìm hiểu trong quá trình học, việc kết hợp chúng trong một chương trình thực tế giúp nhóm hiểu rõ hơn mối liên hệ giữa các thành phần. Ví dụ, khi nhiều luồng cùng xử lý dữ liệu của một trận đấu, hệ thống cần có cơ chế đồng bộ phù hợp để tránh phát sinh xung đột hoặc làm sai lệch trạng thái bàn cờ.

Về mặt thực tiễn, sản phẩm có thể hoạt động trong mạng LAN, cho phép các máy tính kết nối với nhau để trải nghiệm và kiểm thử những chức năng của trò chơi. Trong quá trình trình diễn, server hiển thị địa chỉ IP của máy đang chạy, giúp người dùng thuận tiện hơn khi thiết lập kết nối từ các máy khác. Nhờ đó, nhóm có thể kiểm tra hoạt động của hệ thống trong môi trường nhiều máy tính mà không cần thực hiện quá nhiều thao tác cấu hình thủ công.

1.2.3. Mục tiêu đề tài

Đề tài hướng đến việc xây dựng một server trung tâm có khả năng tiếp nhận và quản lý nhiều kết nối cùng lúc, trong đó mỗi kết nối được xử lý trên một luồng riêng. Server đóng vai trò quản lý chính của trận đấu, chịu trách nhiệm kiểm tra tính hợp lệ của nước đi, xác định lượt chơi và xử lý các kết quả thắng, thua hoặc hòa. Trong khi đó, client đảm nhận việc hiển thị giao diện và gửi yêu cầu đến server.

Bên cạnh chức năng chơi cờ cơ bản, hệ thống còn hỗ trợ người dùng đăng ký, đăng nhập và tham gia sảnh chờ. Người chơi có thể tạo phòng riêng bằng mã phòng hoặc lựa chọn ghép trận ngẫu nhiên để tìm đối thủ. Những chức năng này giúp việc tham gia trò chơi thuận tiện hơn và tạo nên trải nghiệm gần với một trò chơi trực tuyến hoàn chỉnh.

Ngoài ra, nhóm sử dụng MySQL để lưu trữ thông tin tài khoản và kết quả các trận đấu, phục vụ cho việc xem lại lịch sử chơi và theo dõi bảng xếp hạng. Hệ thống cũng được bổ sung chế độ chơi với máy tính gồm ba mức độ AI, giúp người chơi có thể luyện tập ngay cả khi không có đối thủ trực tuyến. Thông qua các mục tiêu trên, nhóm mong muốn xây dựng một trò chơi Caro có những chức năng tương đối đầy đủ, đồng thời vận dụng được các kiến thức quan trọng về lập trình mạng đã học.

1.2.4. Đối tượng và phạm vi giới hạn

Đối tượng sử dụng của hệ thống là những người muốn chơi cờ Caro thông qua ứng dụng. Tất cả người dùng đều có các quyền cơ bản như tham gia chơi, xem lịch sử trận đấu, theo dõi bảng xếp hạng và chỉnh sửa thông tin hồ sơ của chính mình. Trong phạm vi đồ án, hệ thống chưa xây dựng cơ chế phân quyền dành riêng cho quản trị viên.

Về phạm vi triển khai, ứng dụng được phát triển dưới dạng phần mềm desktop bằng Java Swing, hỗ trợ chạy trên cùng một máy tính hoặc kết nối giữa các máy trong mạng LAN. Do giới hạn về thời gian và phạm vi thực hiện, nhóm chưa phát triển phiên

bản web hoặc ứng dụng di động, chưa triển khai server trên Internet và chưa bổ sung cơ chế mã hóa dữ liệu truyền qua mạng. Những hạn chế khác của hệ thống sẽ được trình bày cụ thể hơn tại mục 3.4.3 của báo cáo.

1.3. Cấu trúc đồ án

Ngoài phần mở đầu, tài liệu tham khảo và phụ lục, báo cáo được chia thành bốn chương. Chương 1 giới thiệu đề tài, trình bày lý do lựa chọn, nhiệm vụ, mục tiêu, đối tượng và phạm vi thực hiện đồ án. Chương 2 trình bày cơ sở lý thuyết được sử dụng trong quá trình xây dựng hệ thống, bao gồm mô hình Client-Server, TCP Socket, lập trình đa luồng, InetAddress, truyền đối tượng qua mạng, JDBC và Java Swing.

Chương 3 tập trung vào quá trình phân tích, thiết kế, cài đặt và kiểm thử hệ thống. Đây là phần trọng tâm của báo cáo, trình bày cách nhóm xây dựng các thành phần và triển khai những chức năng của trò chơi. Cuối cùng, Chương 4 tổng kết kết quả đạt được, đánh giá những hạn chế còn tồn tại và đề xuất một số hướng phát triển cho hệ thống trong tương lai.

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT

2.1. Mô hình Client-Server

Mô hình Client-Server là mô hình tổ chức hệ thống trong đó server chịu trách nhiệm tiếp nhận và xử lý yêu cầu, còn client chủ động kết nối đến server để gửi dữ liệu và nhận kết quả phản hồi. Trong các trò chơi trực tuyến, mô hình này có ưu điểm là giúp quản lý trạng thái trận đấu tại một nơi thống nhất. Nếu mỗi client tự lưu bàn cờ và chỉ trao đổi nước đi trực tiếp với nhau theo mô hình ngang hàng, sự chậm trễ trong quá trình truyền dữ liệu có thể khiến trạng thái bàn cờ giữa hai bên không còn khớp nhau. Khi đó, việc xác định trạng thái nào là chính xác cũng trở nên khó khăn hơn.

Với mô hình Client-Server, server đóng vai trò trung tâm trong việc quản lý và xác nhận các hoạt động của trận đấu. Mọi nước đi đều được gửi đến server để kiểm tra trước khi cập nhật trạng thái chung và thông báo cho các client liên quan. Nhờ vậy, hai người chơi có thể theo dõi cùng một bàn cờ và hạn chế những sai lệch phát sinh trong quá trình trao đổi dữ liệu. Đây cũng là lý do nhóm lựa chọn mô hình này cho đồ án. Ngoài việc đồng bộ trạng thái trận đấu, cách tổ chức này còn giúp hạn chế những hành vi gian lận cơ bản, chẳng hạn như client tự ý thông báo mình đã chiến thắng mà chưa được server xác nhận.

2.2. TCP Socket trong Java

TCP (Transmission Control Protocol) là giao thức truyền dữ liệu có hướng kết nối. Trước khi trao đổi dữ liệu, hai phía cần thiết lập kết nối với nhau. Sau đó, TCP bảo đảm dữ liệu được truyền đến theo đúng thứ tự và cung cấp cơ chế phát hiện, truyền lại dữ liệu khi xảy ra mất mát trong quá trình truyền. Đối với trò chơi Caro trực tuyến, việc nhận dữ liệu đúng thứ tự rất quan trọng vì mỗi nước đi đều phụ thuộc vào trạng thái của bàn cờ trước đó. Nếu thứ tự xử lý các nước đi bị sai lệch, trạng thái trận đấu có thể không còn chính xác. Vì vậy, nhóm lựa chọn TCP thay vì UDP, vốn không tự bảo đảm việc truyền dữ liệu đầy đủ và đúng thứ tự.

Trong Java, lớp ServerSocket được sử dụng ở phía server để lắng nghe các yêu cầu kết nối trên một cổng xác định. Khi có client kết nối đến, phương thức accept() trả về một đối tượng Socket đại diện cho kết nối đó. Ở phía client, chương trình sử dụng Socket(ip, port) để thiết lập kết nối đến địa chỉ IP và cổng của server. Sau khi kết nối thành công, hai phía có thể lấy các luồng nhập và xuất từ đối tượng Socket để trao đổi dữ liệu. Đây là cơ chế giao tiếp mạng cơ bản được nhóm sử dụng trong quá trình xây dựng trò chơi.

2.3. Lập trình đa luồng (Thread) trong Java

Trong lập trình mạng, một số thao tác như accept() và readObject() có thể khiến luồng thực thi phải chờ cho đến khi có kết nối hoặc dữ liệu được gửi đến. Nếu server chỉ sử dụng một luồng để xử lý tất cả các client, quá trình chờ dữ liệu từ một kết nối có thể làm chậm việc phục vụ những kết nối khác. Điều này sẽ ảnh hưởng đến khả năng hỗ trợ nhiều người chơi cùng lúc. Để giải quyết vấn đề trên, nhóm sử dụng cơ chế đa luồng

trong Java. Luồng chính đảm nhận việc tiếp nhận các kết nối mới, sau đó giao từng kết nối cho một luồng riêng xử lý. Nhờ đó, server có thể tiếp tục nhận kết nối trong khi các client hiện tại vẫn đang trao đổi dữ liệu.

Tuy nhiên, khi nhiều luồng cùng hoạt động, việc truy cập dữ liệu dùng chung cần được kiểm soát để tránh xảy ra xung đột. Chẳng hạn, hai người chơi trong cùng một trận đấu có thể gửi yêu cầu gần như đồng thời. Nếu server không xử lý phù hợp, cả hai yêu cầu có thể cùng tác động đến một ô trên bàn cờ. Java cung cấp từ khóa synchronized để kiểm soát những đoạn mã cần được thực thi đồng bộ. Bên cạnh đó, ConcurrentHashMap hỗ trợ quản lý các tập dữ liệu được nhiều luồng truy cập, chẳng hạn như thông tin phòng chơi. Trong đồ án, nhóm cũng sử dụng java.util.Timer để hỗ trợ việc đếm thời gian cho mỗi lượt đi.

2.4. Lớp InetAddress

InetAddress là lớp trong Java hỗ trợ biểu diễn và xử lý địa chỉ IP, đồng thời cung cấp các phương thức phân giải tên máy thành địa chỉ IP. Trong đồ án, nhóm kết hợp lớp này với NetworkInterface để lấy thông tin địa chỉ mạng của máy tính đang chạy server và hiển thị các địa chỉ IPv4 phù hợp khi chương trình khởi động.

Chức năng này giúp người dùng thuận tiện hơn trong quá trình kết nối các máy tính với nhau, đặc biệt khi trình diễn hoặc kiểm thử ứng dụng trong mạng LAN. Thay vì phải tự tìm địa chỉ IP của máy chủ bằng các công cụ bên ngoài, người dùng có thể xem thông tin được chương trình hiển thị và sử dụng địa chỉ đó để thiết lập kết nối từ client. Dù không phải thành phần cốt lõi của cơ chế chơi cờ, việc sử dụng InetAddress góp phần đơn giản hóa quá trình triển khai và kiểm thử hệ thống.

2.5. Truyền đối tượng qua luồng nhập/xuất mạng

Trong Java, dữ liệu có thể được trao đổi qua mạng dưới dạng đối tượng bằng cách sử dụng ObjectOutputStream và ObjectInputStream. So với việc tự xây dựng định dạng chuỗi rồi phân tích từng phần khi nhận dữ liệu, phương pháp này cho phép chương trình gửi và nhận các đối tượng Java trực tiếp. Để thực hiện được điều đó, lớp của đối tượng cần triển khai giao diện Serializable. Ngoài ra, các lớp tương ứng phải được cung cấp ở cả phía gửi và phía nhận để chương trình có thể chuyển đổi dữ liệu thành đối tượng phù hợp.

Trong quá trình xây dựng hệ thống, nhóm đặt các lớp dùng chung cho việc trao đổi dữ liệu vào package caro.common nhằm giúp server và client sử dụng thống nhất cấu trúc đối tượng. Nếu lớp cần thiết không tồn tại ở phía nhận hoặc thông tin phiên bản lớp không tương thích, chương trình có thể phát sinh các ngoại lệ như ClassNotFoundException hoặc InvalidClassException.

Bên cạnh đó, nhóm cũng cần chú ý đến cách khởi tạo và sử dụng các luồng đối tượng. Khi tạo ObjectInputStream, chương trình sẽ đọc phần header được ghi ở đầu luồng dữ liệu. Vì vậy, nếu cả hai phía đều khởi tạo luồng nhập trước khi tạo luồng xuất, chúng có thể chờ lẫn nhau và khiến kết nối bị treo. Để tránh tình huống này, nhóm thực

hiện tạo ObjectOutputStream và gọi flush() trước, sau đó mới khởi tạo ObjectInputStream ở mỗi phía.

Một vấn đề khác liên quan đến cơ chế lưu tham chiếu đối tượng của ObjectOutputStream. Khi một đối tượng đã được gửi trước đó tiếp tục được gửi lại sau khi nội dung thay đổi, luồng có thể sử dụng thông tin tham chiếu đã lưu thay vì ghi lại toàn bộ trạng thái mới của đối tượng. Điều này có thể khiến phía nhận không cập nhật được dữ liệu như mong muốn. Để hạn chế vấn đề trên, nhóm sử dụng phương thức reset() sau khi gửi dữ liệu trong những trường hợp cần thiết, giúp làm mới thông tin tham chiếu mà luồng đã lưu.

2.6. JDBC và MySQL

JDBC (Java Database Connectivity) là API của Java hỗ trợ kết nối và thao tác với các hệ quản trị cơ sở dữ liệu quan hệ. Thông qua JDBC, chương trình có thể thực hiện các thao tác như truy vấn, thêm mới, cập nhật và xóa dữ liệu. Trong đồ án, nhóm sử dụng trình điều khiển mysql-connector-j phiên bản 8.0.33 để kết nối ứng dụng Java với MySQL 5.7 được cài đặt thông qua WampServer.

Thay vì sử dụng các công cụ ánh xạ đối tượng – quan hệ (ORM) như Hibernate, nhóm lựa chọn viết câu lệnh SQL trực tiếp thông qua JDBC. Cách làm này giúp các thành viên hiểu rõ hơn quá trình truy vấn và cập nhật dữ liệu, đồng thời chủ động kiểm soát những thao tác được thực hiện trên cơ sở dữ liệu. Trong quá trình xây dựng các chức năng liên quan đến tài khoản và lịch sử trận đấu, nhóm sử dụng PreparedStatement để truyền các tham số vào câu lệnh SQL. Cách tiếp cận này giúp hạn chế nguy cơ SQL Injection khi dữ liệu đầu vào được sử dụng trong truy vấn.

2.7. Java Swing

Java Swing là thư viện được sử dụng để xây dựng giao diện đồ họa cho các ứng dụng Java desktop. Trong đồ án, nhóm sử dụng Swing để thiết kế giao diện trò chơi, bao gồm các màn hình chức năng, bàn cờ và những thành phần tương tác dành cho người chơi. Một số chi tiết giao diện như nút bo góc, cách hiển thị bàn cờ và hiệu ứng pháo giấy khi chiến thắng được nhóm tự vẽ bằng Graphics2D, thay vì sử dụng thư viện giao diện bên ngoài. Việc này giúp nhóm chủ động hơn trong việc điều chỉnh hình thức hiển thị theo yêu cầu của trò chơi.

Khi phát triển ứng dụng bằng Swing, một nguyên tắc quan trọng là các thao tác cập nhật giao diện cần được thực hiện trên luồng xử lý sự kiện của giao diện (Event Dispatch Thread – EDT). Nếu thực hiện các thao tác chờ dữ liệu mạng trực tiếp trên luồng này, giao diện có thể bị đơ trong thời gian chờ phản hồi từ server. Vì vậy, nhóm tách các công việc giao tiếp mạng sang luồng riêng, sau đó chuyển những thao tác cập nhật giao diện về EDT khi cần thiết. Cách tổ chức này giúp giao diện phản hồi tốt hơn trong lúc người chơi thực hiện các thao tác hoặc chờ dữ liệu từ server.

CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM

3.1. Yêu cầu hệ thống

Về chức năng, hệ thống cho phép người dùng đăng ký và đăng nhập bằng tài khoản cá nhân. Sau khi đăng nhập, người chơi được chuyển đến sảnh chờ, tại đây có thể tạo phòng riêng và nhận mã phòng gồm 6 ký tự để chia sẻ với bạn bè, tham gia phòng bằng mã có sẵn, xem danh sách các phòng đang chờ hoặc sử dụng chức năng "Chơi Online" để tìm đối thủ ngẫu nhiên. Khi trận đấu bắt đầu, hai người chơi lần lượt đánh cờ trên bàn 15×15, với thời gian tối đa 20 giây cho mỗi lượt. Trong quá trình chơi, hệ thống hỗ trợ trò chuyện, đầu hàng, đề nghị hòa và chơi lại nếu cả hai bên cùng đồng ý. Ngoài chế độ trực tuyến, người chơi còn có thể đấu với máy ở ba mức độ Dễ, Trung bình và Khó, xem lịch sử trận đấu, theo dõi bảng xếp hạng và thay đổi tên hiển thị của mình.

Bên cạnh các chức năng trên, hệ thống cũng cần đáp ứng một số yêu cầu về độ ổn định và an toàn. Nhiều trận đấu phải có khả năng diễn ra đồng thời mà không ảnh hưởng đến nhau. Khi một người chơi mất kết nối, server vẫn phải hoạt động bình thường và người chơi còn lại không bị treo chương trình. Tương tự, nếu xảy ra lỗi khi ghi dữ liệu vào cơ sở dữ liệu, trận đấu đang diễn ra vẫn phải tiếp tục được xử lý. Đối với thông tin tài khoản, mật khẩu không được lưu trực tiếp dưới dạng văn bản thông thường trong database.

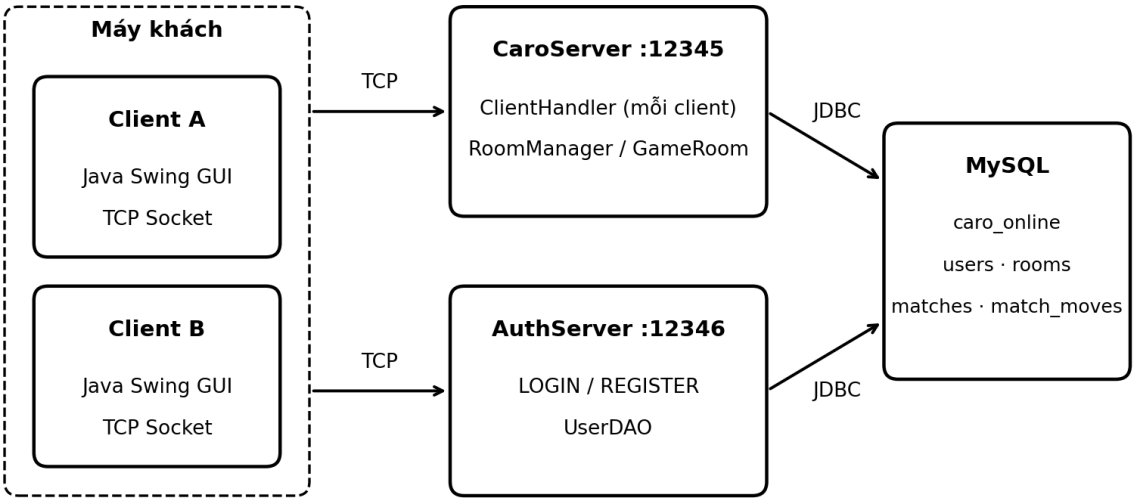
3.2. Phân tích và thiết kế hệ thống

3.2.1. Kiến trúc tổng thể

Hệ thống Caro Online được xây dựng từ ba thành phần chính: server, client và cơ sở dữ liệu MySQL. Trong quá trình thiết kế, nhóm sử dụng hai cổng TCP riêng biệt để tách phần xác thực tài khoản khỏi phần quản lý sảnh chờ và trận đấu.

Cụ thể, cổng 12346 do AuthServer quản lý, chịu trách nhiệm xử lý các yêu cầu đăng nhập và đăng ký. Kết nối qua cổng này chỉ tồn tại trong thời gian ngắn: client gửi yêu cầu, server kiểm tra và trả về kết quả, sau đó kết nối được đóng lại. Trong khi đó, cổng 12345 do CaroServer quản lý, đảm nhận các hoạt động diễn ra sau khi người dùng đăng nhập, bao gồm tham gia sảnh chờ, tạo phòng và chơi cờ. Mỗi client duy trì một kết nối trong suốt thời gian ở sảnh hoặc tham gia trận đấu, đồng thời được giao cho một đối tượng ClientHandler xử lý trên luồng riêng.

Ban đầu, nhóm dự định sử dụng chung một cổng cho cả đăng nhập và chơi cờ. Tuy nhiên, khi xem xét cách server quản lý kết nối, nhóm nhận thấy những kết nối chỉ dùng để xác thực có thể bị xử lý nhầm như kết nối của người đang chờ ghép trận. Điều này có nguy cơ làm ảnh hưởng đến cơ chế ghép phòng đã được xây dựng. Vì vậy, nhóm quyết định tách thành hai cổng để phân định rõ trách nhiệm của từng phần và hạn chế tác động đến các chức năng hiện có.



Hình 3.1. Sơ đồ kiến trúc tổng thể hệ thống Caro Online.

Ở giữa tầng mạng và cơ sở dữ liệu là các thành phần RoomManager, GameRoom cùng những lớp DAO. Trong đó, RoomManager được thiết kế theo mô hình singleton, sử dụng ConcurrentHashMap để quản lý danh sách phòng chơi. Cách tổ chức này cho phép nhiều phòng được quản lý đồng thời. Mỗi GameRoom phụ trách một trận đấu cụ thể, từ việc lưu trạng thái bàn cờ, kiểm tra lượt đi đến quản lý thời gian và xác định kết quả thắng, thua hoặc hòa. Khi trận đấu kết thúc, GameRoom gọi MatchDAO và UserDAO để lưu kết quả vào cơ sở dữ liệu.

Chế độ chơi với máy được tổ chức theo cách khác so với chế độ trực tuyến. Do chỉ có một người chơi tương tác với AI trên máy tính của mình, chế độ này được xử lý hoàn toàn ở phía client và không cần thông qua server để điều phối nước đi. Tuy vậy, kết quả trận đấu vẫn được lưu vào MySQL thông qua các lớp DAO để phục vụ việc thống kê và xem lịch sử.

3.2.2. Thiết kế lớp

Để mã nguồn dễ quản lý và thuận tiện cho việc phát triển, nhóm chia hệ thống thành nhiều package theo chức năng. Package caro.common chứa các lớp dùng chung giữa client và server, gồm Message, User, RoomInfo và MatchSummary. Những lớp này đều triển khai Serializable để phục vụ việc truyền đối tượng qua mạng. Riêng lớp User không chứa thông tin mật khẩu, nhằm tránh đưa dữ liệu nhạy cảm vào các đối tượng tài khoản được trao đổi giữa hai phía.

Package caro.server tập trung vào các thành phần xử lý phía server. Trong đó, CaroServer là điểm khởi động chính, còn ServerFrame cung cấp cửa sổ giao diện để theo dõi server khi trình diễn. Các lớp AuthServer và AuthService đảm nhận phần xác thực tài khoản; RoomManager quản lý phòng và ghép trận; ClientHandler xử lý kết nối của từng client; còn GameRoom chịu trách nhiệm điều phối hoạt động trong mỗi ván cờ. Đối với cơ sở dữ liệu, package caro.database gồm DatabaseConfig,

DatabaseConnection, UserDAO, MatchDAO và MoveDAO, giúp tách các thao tác lưu trữ dữ liệu khỏi phần xử lý trò chơi.

Bên cạnh đó, package caro.ai chứa interface CaroAI cùng ba lớp triển khai là EasyAI, MediumAI và HardAI. Phần giao diện được tổ chức trong hai package caro.client và caro.ui. Các màn hình chính bao gồm đăng nhập, đăng ký, sảnh chờ, phòng chờ, bàn cờ trực tuyến thông qua CaroClient, bàn cờ với máy thông qua AIGameFrame, bảng xếp hạng, lịch sử trận đấu và hồ sơ người dùng. Một số thành phần giao diện dùng chung như Theme, RoundedButton và RoundedPanel cũng được đặt riêng để thuận tiện khi điều chỉnh cách hiển thị.

Đối với AI, cả ba mức độ đều triển khai interface CaroAI, nhờ đó màn hình chơi với máy không cần xử lý riêng cho từng cấp độ. Sự khác biệt chủ yếu nằm ở cách lựa chọn nước đi. EasyAI chọn nước đi ngẫu nhiên. MediumAI ưu tiên những nước có thể giúp máy thắng ngay, tiếp theo là chặn nước thắng của đối thủ; nếu chưa có tình huống nào như vậy, máy sẽ chọn ô có khả năng tạo thành chuỗi quân dài nhất. Trong khi đó, HardAI đánh giá từng ô trống dựa trên các mẫu quân cờ, kết hợp điểm tấn công của máy với điểm phòng thủ trước đối thủ. Điểm phòng thủ được nhân thêm hệ số 1,05 để AI có xu hướng ưu tiên ngăn chặn đối phương hơn một chút.

Để giảm số lượng ô cần đánh giá, HardAI chỉ xét những ô nằm trong bán kính hai ô tính từ các quân cờ đã xuất hiện trên bàn. Nhóm lựa chọn phương pháp heuristic thay vì Minimax vì bàn cờ 15×15 có tổng cộng 225 ô. Việc tìm kiếm trên toàn bộ các khả năng có thể làm tăng đáng kể thời gian xử lý, trong khi phương pháp heuristic phù hợp hơn với phạm vi đồ án và cũng thuận tiện khi giải thích nguyên lý hoạt động của AI.

3.2.3. Thiết kế giao thức trao đổi dữ liệu

Client và server trao đổi dữ liệu thông qua lớp Message thống nhất. Mỗi thông điệp có trường type để xác định loại yêu cầu hoặc phản hồi, còn các trường dữ liệu khác được sử dụng tùy theo từng loại thông điệp. Cách thiết kế này giúp việc trao đổi thông tin giữa hai phía nhất quán và dễ quản lý hơn.

Trường	Kiểu	Ý nghĩa
type	Message.Type	Loại thông điệp
playerId	int	1 hoặc 2, người chơi nào trong ván
x, y	int	Tọa độ ô cờ (dùng cho MOVE)
nextTurn	int	Người được đánh tiếp theo
winnerId	int	Người thắng (dùng cho WIN)
note	String	Tên người chơi, thông báo, hoặc nội dung chat
winLineX, winLineY	int[]	Các ô nằm trên đường thắng để tô sáng

Trường	Kiểu	Ý nghĩa
username, password, displayName	String	Dùng cho đăng nhập, đăng ký, đổi tên
user	User	Thông tin tài khoản trả về
roomCode	String	Mã phòng
roomList	List<RoomInfo>	Danh sách phòng đang chờ
matchHistory	List<MatchSummary>	Lịch sử trận đấu
leaderboard	List<User>	Bảng xếp hạng

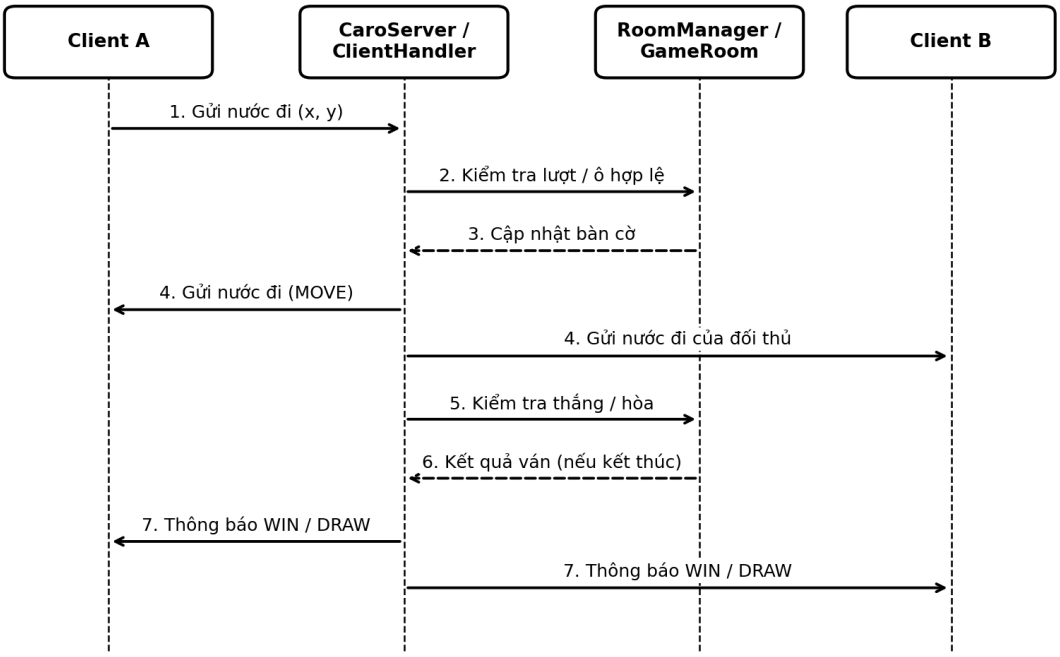
Bảng 3.1. Mô tả các trường dữ liệu của lớp Message.

Nhóm	Thông điệp	Hướng
Xác thực (cổng 12346)	LOGIN, REGISTER	Client → Server
	LOGIN_SUCCESS, LOGIN_FAILED, REGISTER_SUCCESS, REGISTER_FAILED	Server → Client
Sảnh và phòng	CREATE_ROOM, JOIN_ROOM, LEAVE_ROOM, ROOM_LIST, FIND_MATCH, CANCEL_FIND	Client → Server
	ROOM_CREATED, ROOM_JOINED, ROOM_JOIN_FAILED, WAITING, ROOM_LIST	Server → Client
Trong ván	NAME, MOVE, CHAT, REPLAY, SURRENDER, OFFER_DRAW, DRAW_ACCEPT, DRAW_REJECT	Client → Server
	START, MOVE, WIN, DRAW, DRAW_REJECT, OPPONENT_LEFT, REPLAY, CHAT, OFFER_DRAW, NAME	Server → Client
Thống kê và hồ sơ	MATCH_HISTORY, LEADERBOARD, PROFILE_UPDATE	Client → Server
	PROFILE_UPDATED, ERROR, MATCH_HISTORY, LEADERBOARD	Server → Client

Bảng 3.2. Các loại thông điệp trong giao thức

Khi người chơi thực hiện nước đi, client gửi MOVE kèm tọa độ đến server để kiểm tra lượt chơi, vị trí và ô cờ. Nếu hợp lệ, server đặt quân, lưu nước đi vào database rồi

gửi MOVE cùng nextTurn cho cả hai bên. Nước đi không hợp lệ sẽ bị bỏ qua. Client chỉ cập nhật bàn cờ sau khi nhận phản hồi từ server, đảm bảo hai bên luôn đồng bộ.



Hình 3.2. Sơ đồ tuần tự một lượt đánh trong ván online

Một lượt đánh trong chế độ trực tuyến bắt đầu khi client gửi thông điệp MOVE kèm theo tọa độ ô cờ. Server tiếp nhận yêu cầu rồi kiểm tra ba điều kiện: người gửi có đang đến lượt hay không, tọa độ có nằm trong phạm vi bàn cờ hay không và ô được chọn đã có quân hay chưa. Chỉ khi tất cả điều kiện đều hợp lệ, server mới cập nhật bàn cờ, lưu nước đi vào cơ sở dữ liệu và gửi thông điệp MOVE cho cả hai người chơi, kèm theo thông tin nextTurn để xác định lượt tiếp theo.

Nếu nước đi không hợp lệ, server bỏ qua yêu cầu và không gửi phản hồi. Phía client cũng không tự cập nhật quân cờ ngay khi người chơi nhấn vào một ô, mà chỉ hiển thị nước đi sau khi nhận được xác nhận từ server. Nhờ cách xử lý này, server giữ vai trò kiểm soát trạng thái chung, đồng thời hạn chế tình trạng hai client hiển thị bàn cờ khác nhau.

3.2.4. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu caro_online sử dụng MySQL và bộ mã hóa utf8mb4 để hỗ trợ lưu trữ tiếng Việt có dấu. Hệ thống gồm bốn bảng chính. Bảng users lưu thông tin tài khoản, thống kê số trận thắng, thua, hòa và điểm rating của người chơi. Bảng matches ghi lại thông tin từng ván đấu, bao gồm hai người chơi, người thắng, kết quả, chế độ chơi (ONLINE hoặc AI), độ khó của AI cùng thời điểm bắt đầu và kết thúc. Trong khi đó, bảng match_moves lưu lại từng nước đi theo thứ tự, kèm thời gian thực hiện chính xác đến mili giây. Cuối cùng, bảng rooms dùng để lưu trữ thông tin các phòng chơi trong hệ thống.

Cột	Kiểu	Ghi chú
id	INT UNSIGNED, khóa chính	Tự tăng
username	VARCHAR(32)	Duy nhất
password	VARCHAR(64)	Chuỗi SHA-256 dạng hex, không lưu chữ thường
display_name	VARCHAR(64)	Tên hiển thị, cho phép đổi
wins, losses, draws, total_games	INT UNSIGNED	Thống kê, mặc định 0
rating	INT	Mặc định 1000, có chỉ mục để sắp bảng xếp hạng
created_at, last_login	DATETIME	Thời điểm tạo và đăng nhập gần nhất
status	ENUM	ONLINE, IN_LOBBY, IN_ROOM, PLAYING, OFFLINE

Bảng 3.3. Cấu trúc bảng users

Cột	Kiểu dữ liệu	Khóa / ràng buộc	Ý nghĩa
id	INT UNSIGNED	PK, AUTO_INCREMENT	Mã phòng nội bộ
room_code	VARCHAR(10)	UNIQUE, NOT NULL	Mã phòng
host_id	INT UNSIGNED	FK → users.id, NOT NULL	Người tạo phòng
guest_id	INT UNSIGNED	FK → users.id, NULL	Người tham gia
status	ENUM	NOT NULL, DEFAULT WAITING	WAITING, PLAYING, FINISHED
mode	ENUM	NOT NULL, DEFAULT ONLINE	ONLINE hoặc AI
created_at	DATETIME	NOT NULL	Thời điểm tạo phòng

Bảng 3.4. Cấu trúc bảng rooms

Cột	Kiểu dữ liệu	Khóa / ràng buộc	Ý nghĩa
id	INT UNSIGNED	PK, AUTO_INCREMENT	Mã ván đấu

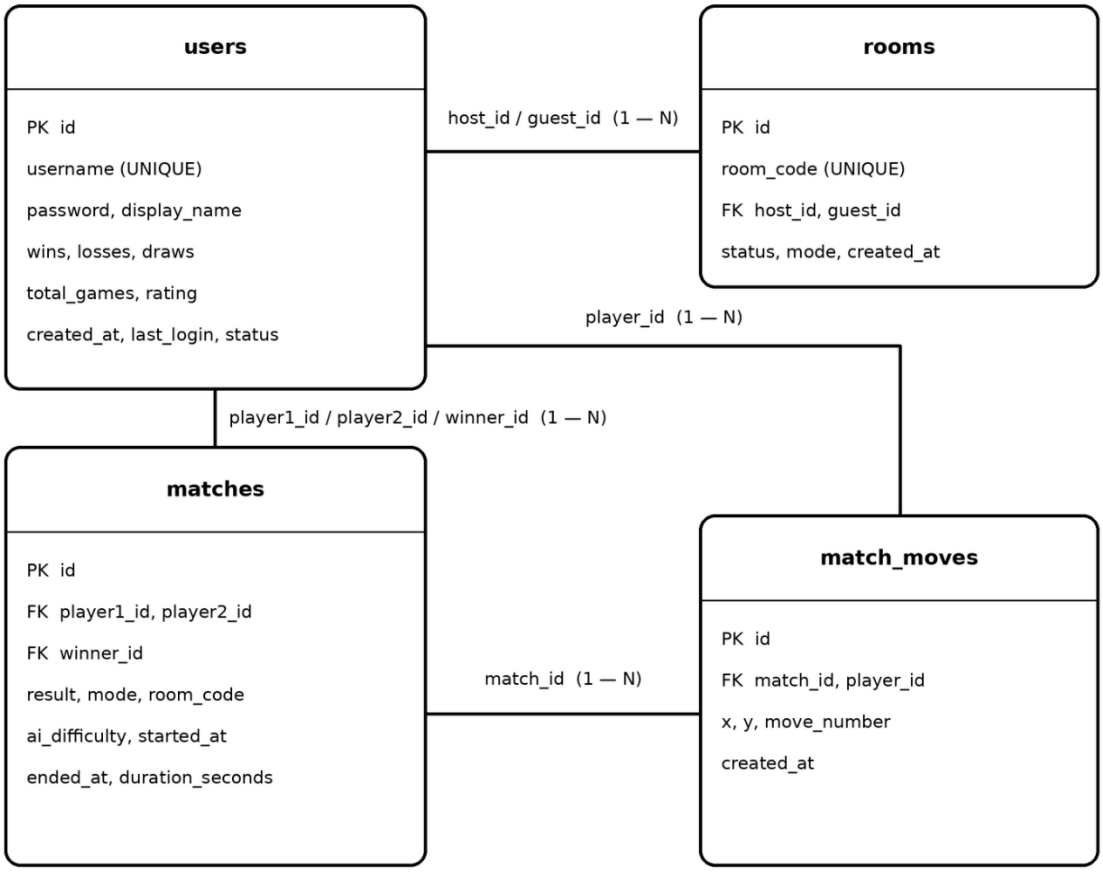
Cột	Kiểu dữ liệu	Khóa / ràng buộc	Ý nghĩa
player1_id	INT UNSIGNED	FK → users.id, NOT NULL	Người chơi 1
player2_id	INT UNSIGNED	FK → users.id, NULL	NULL khi chơi với AI
winner_id	INT UNSIGNED	FK → users.id, NULL	Người thắng; NULL nếu hòa hoặc chưa xác định
result	ENUM	NOT NULL	WIN, LOSS, DRAW, ABANDONED
mode	ENUM	NOT NULL, DEFAULT ONLINE	ONLINE hoặc AI
room_code	VARCHAR(10)	NULL	Mã phòng liên quan
ai_difficulty	ENUM	NULL	EASY, MEDIUM, HARD; dùng cho AI
started_at / ended_at	DATETIME	NOT NULL / NULL	Thời điểm bắt đầu / kết thúc
duration_seconds	INT UNSIGNED	NULL	Thời lượng ván (giây)

Bảng 3.5. Cấu trúc bảng matches

Cột	Kiểu dữ liệu	Khóa / ràng buộc	Ý nghĩa
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Mã nước đi
match_id	INT UNSIGNED	FK → matches.id, NOT NULL	Ván đấu chứa nước đi
player_id	INT UNSIGNED	FK → users.id, NOT NULL	Người thực hiện nước đi
x / y	TINYINT UNSIGNED	NOT NULL	Tọa độ ô trên bàn cờ
move_number	SMALLINT UNSIGNED	NOT NULL	Thứ tự nước đi
created_at	DATETIME(3)	NOT NULL	Thời điểm lưu nước đi

Bảng 3.6. Cấu trúc bảng match_moves

Các khóa ngoại nối matches và match_moves về users. Khi xóa một ván thì các nước đi của ván đó xóa theo (ON DELETE CASCADE).



Hình 3.3. Sơ đồ quan hệ giữa các bảng trong cơ sở dữ liệu.

3.3. Cài đặt chương trình

3.3.1. Môi trường phát triển

Thành phần	Chi tiết
Ngôn ngữ	Java 8 (javac.source và javac.target đều là 1.8)
IDE và công cụ build	NetBeans IDE 8.0.2, Ant
Giao diện	Java Swing, tự vẽ bằng Graphics2D
Mạng	TCP Socket (java.net.Socket, ServerSocket)
Truyền dữ liệu	Java Object Serialization
Cơ sở dữ liệu	MySQL 5.7.44 chạy qua WampServer
Kết nối DB	JDBC thuần, mysql-connector-j 8.0.33

3.3.2. Cấu trúc thư mục project

Mã nguồn đầy đủ của project được đăng tại GitHub của nhóm: https://github.com/nhungoc260/caro-online-java [9].

```
CaroOnline/
├── database/caro_online.sql     (schema và dữ liệu demo)
├── src/caro/
```

```
│   ├── common/    Message, User, RoomInfo, MatchSummary
│   ├── database/  DatabaseConfig, DatabaseConnection, UserDAO, MatchDAO, MoveDAO,
TestDB
│   ├── server/    CaroServer, ServerFrame, AuthServer, AuthService,
│   │              RoomManager, ClientHandler, GameRoom
│   ├── ai/        CaroAI, EasyAI, MediumAI, HardAI
│   ├── ui/        Theme, RoundedButton, RoundedPanel
│   └── client/    ConnectFrame, LoginFrame, RegisterFrame, LobbyFrame, RoomFrame,
│                  CaroClient, AIGameFrame, LeaderboardFrame, HistoryFrame,
│                  ProfileFrame, NetUtil
├── dist/          CaroOnline.jar và thư mục lib chứa driver MySQL
└── nbproject/, build.xml
```

3.3.3. Một số đoạn mã nguồn tiêu biểu

a) Vòng lặp nhận kết nối của server

Đây là phần thể hiện rõ nhất cơ chế mỗi client được xử lý bằng một luồng riêng. Luồng chính liên tục chờ kết nối mới, sau đó tạo một ClientHandler để phụ trách client đó và khởi chạy luồng tương ứng. Nhờ vậy, server có thể phục vụ nhiều client cùng lúc mà không phải xử lý tất cả trên một luồng duy nhất.

```
while (true) {
    Socket socket = serverSocket.accept();
    ClientHandler handler = new ClientHandler(socket);
    new Thread(handler).start();
}
```

b) Vòng lặp đọc thông điệp của một client

Khi client kết nối đến server, luồng gửi dữ liệu ObjectOutputStream được khởi tạo trước và gọi flush() ngay sau đó. Tiếp theo, server mới tạo luồng nhận dữ liệu ObjectInputStream. Thứ tự này giúp tránh tình trạng hai bên chờ header của nhau, khiến kết nối bị treo như đã đề cập ở mục 2.5. Sau khi thiết lập xong, server sử dụng một vòng lặp duy nhất để nhận và xử lý thông điệp. Dù client đang ở sảnh chờ hay trong một ván đấu, các thông điệp đều được chuyển đến handle() để xử lý theo từng trường hợp.

```
out = new ObjectOutputStream(socket.getOutputStream());
out.flush();
in = new ObjectInputStream(socket.getInputStream());

Message msg;
while ((msg = (Message) in.readObject()) != null) {
    handle(msg);
}
```

c) Xử lý một nước đi

Phương thức handleMove() được khai báo với từ khóa synchronized nhằm tránh việc hai luồng cùng thay đổi trạng thái bàn cờ tại một thời điểm. Trước khi chấp nhận nước đi, server lần lượt kiểm tra ván đấu đã kết thúc chưa, người chơi có đúng lượt không, tọa độ có nằm trong phạm vi bàn cờ không và ô được chọn đã có quân hay chưa. Nếu vi phạm bất kỳ điều kiện nào, nước đi sẽ bị bỏ qua. Chỉ khi vượt qua tất cả các bước kiểm tra, nước đi mới được thực hiện thông qua applyMove().

```
public synchronized void handleMove(int playerId, int x, int y) {
    if (gameOver) return;
    if (playerId != currentTurn) return;
    if (x < 0 || y < 0 || x >= SIZE || y >= SIZE) return;
```

```
    if (board[x][y] != 0) return;
    applyMove(playerId, x, y);
}
```

d) Kiểm tra điều kiện thắng

Sau mỗi nước đi, server không cần duyệt lại toàn bộ 225 ô trên bàn cờ mà chỉ kiểm tra các quân cùng màu nằm trên những đường đi qua ô vừa đánh. Việc kiểm tra được thực hiện theo bốn hướng: ngang, dọc và hai đường chéo. Server thu thập các ô liên tiếp theo cả hai phía của ô vừa đánh. Nếu tổng số quân trên một đường đạt từ 5 quân trở lên, người chơi được xác định là chiến thắng. Danh sách các ô tạo thành đường thắng cũng được lưu vào lastWinCells để gửi cho client tô sáng trên giao diện.

```
int[][] directions = { {1, 0}, {0, 1}, {1, 1}, {1, -1} };
for (int[] d : directions) {
    List<int[]> line = new ArrayList<int[]>();
    line.add(new int[]{x, y});
    collectDirection(x, y, d[0], d[1], playerId, line);
    collectDirection(x, y, -d[0], -d[1], playerId, line);
    if (line.size() >= 5) { lastWinCells = line; return true; }
}
return false;
```

e) Ghép trận ngẫu nhiên

Cơ chế ghép trận sử dụng hàng đợi chỉ lưu tối đa một người chơi đang chờ. Khi người đầu tiên chọn chơi online, server lưu người đó vào hàng đợi. Ngay khi có người thứ hai tham gia, hai người được ghép thành một cặp và một phòng đấu mới được tạo. Ngoài ra, server kiểm tra điều kiện waitingForMatch == player để tránh trường hợp cùng một người bấm đúp nút Chơi Online, khiến yêu cầu ghép trận bị gửi lặp lại.

3.3.4. Hướng dẫn biên dịch và chạy chương trình

Để chạy chương trình, trước tiên cần khởi động WampServer, sau đó mở phpMyAdmin hoặc dòng lệnh MySQL để thực thi file database/caro_online.sql. File này tạo cơ sở dữ liệu, bốn bảng và một số tài khoản dùng để thử nghiệm, gồm demo1, nguyena và minh, với mật khẩu mặc định là 123456.

Tiếp theo, mở project CaroOnline bằng NetBeans. Nếu project chưa có thư viện kết nối MySQL, hãy thêm file mysql-connector-j-8.0.33.jar vào mục Libraries. Thông tin kết nối cơ sở dữ liệu được cấu hình trong DatabaseConfig.java, mặc định sử dụng tài khoản root, mật khẩu rỗng và địa chỉ 127.0.0.1:3306. Nếu MySQL trên máy được thiết lập mật khẩu khác, cần cập nhật lại thông tin này cho phù hợp.

Sau khi hoàn tất cấu hình, chạy ServerFrame.java hoặc CaroServer.java để khởi động server. Địa chỉ IP của máy sẽ được hiển thị trong cửa sổ Console. Hãy ghi lại địa chỉ này để các máy client sử dụng khi kết nối. Server đồng thời mở hai cổng 12345 và 12346. Nếu xuất hiện thông báo cổng đã được sử dụng, hãy kiểm tra xem server cũ đã dừng chưa; có thể mở tab Output trong NetBeans và nhấn Stop để kết thúc tiến trình đó.

Khi server đã hoạt động, chạy ConnectFrame.java trên từng máy client, nhập địa chỉ IP của server rồi đăng nhập hoặc đăng ký tài khoản. Chỉ cần hai client là có thể bắt

đầu một ván đấu. Nếu chạy cả hai client trên cùng một máy, có thể sử dụng địa chỉ 127.0.0.1.

Ngoài cách chạy trực tiếp trên NetBeans, người dùng cũng có thể build project rồi chạy file CaroOnline.jar bằng lệnh java -jar CaroOnline.jar trong thư mục dist. Khi sử dụng cách này, cần giữ thư mục lib bên cạnh file JAR để chương trình có thể tải các thư viện cần thiết.

3.4. Kiểm thử và kết quả

3.4.1. Kịch bản kiểm thử

Nhóm tiến hành kiểm thử theo hai cách. Thứ nhất, nhóm chạy chương trình trực tiếp trên máy phát triển bằng cách khởi động WampServer, sau đó chạy server và các client để kiểm tra các chức năng như đăng ký, đăng nhập, tạo và tham gia phòng, chơi online, chơi với AI, xem bảng xếp hạng và lịch sử đấu. Trong quá trình này, server và các client cùng chạy trên một máy, kết nối thông qua địa chỉ IP Wi-Fi 192.168.1.6. Kết quả được ghi nhận qua nhật ký chạy chương trình và các hình ảnh minh họa tại mục 3.4.2.

Thứ hai, nhóm sử dụng chương trình kiểm thử riêng gồm hai file T.java và T2.java, được đính kèm trong bài nộp, để thực hiện lại các kịch bản một cách nhất quán. Chương trình mô phỏng hoạt động của client bằng cách đăng nhập qua cổng 12346, kết nối đến cổng 12345 và trao đổi các thông điệp Message giống như khi sử dụng giao diện thật. Nhờ đó, các chức năng được kiểm tra trực tiếp trên server và cơ sở dữ liệu thực tế thay vì môi trường giả lập. Riêng phần AI được kiểm thử thông qua ba lớp EasyAI, MediumAI và HardAI.

Môi trường của lần kiểm thử thứ hai sử dụng JDK 21 và MariaDB 10.11. Mã nguồn vẫn tuân theo chuẩn Java 8, còn MariaDB được sử dụng làm hệ quản trị cơ sở dữ liệu tương thích với giao thức MySQL. Dữ liệu được khởi tạo từ file database/caro_online.sql. Các kịch bản và kết quả kiểm thử được tổng hợp trong Bảng 3.7. Trước khi chạy thử, cơ sở dữ liệu được nạp sẵn ba tài khoản demo là demo1, nguyena và minh.

Kết quả kiểm thử cho thấy cả 19 kịch bản đều đạt yêu cầu đặt ra. Trong đó, ở kịch bản 8, server thực hiện nước đi thay sau khoảng 19–20 giây, sớm hơn một chút so với mốc 20 giây được quy định. Nguyên nhân là đồng hồ phía server bắt đầu đếm ngay khi gửi thông điệp START, trước khi client nhận được thông điệp này. Do đó, thời gian mà người chơi thực tế quan sát được có thể chênh lệch nhẹ so với thời gian được tính ở server.

STT	Kịch bản	Kết quả mong đợi	Kết quả thực tế	Đánh giá
1	Đăng ký tài khoản mới, rồi đăng ký lại với username đã có	Lần đầu thành công; lần hai báo trùng	REGISTER_SUCCESS, sau đó REGISTER_FAILED kèm thông báo tên đăng nhập đã tồn tại	Đạt
2	Đăng nhập với mật khẩu sai	Báo thất bại, không vào được sảnh	LOGIN_FAILED: "Sai tên đăng nhập hoặc mật khẩu."	Đạt
3	Đăng nhập đúng (demo1 / 123456)	Đăng nhập thành công	LOGIN_SUCCESS, rating 1120	Đạt
4	A tạo phòng, B nhập mã để vào	Cả hai nhận START, A cầm X đi trước	Cả hai nhận START; A là người chơi 1, B là người chơi 2	Đạt
5	Nhập mã không tồn tại; vào phòng đang chơi; tự vào phòng của mình	Mỗi trường hợp có một thông báo lỗi riêng	Ba thông báo khác nhau: không tìm thấy phòng; phòng đã đầy hoặc đang chơi; không thể tự vào phòng của mình	Đạt
6	Hai client cùng bấm "Chơi Online"	Được ghép với nhau, tự tạo phòng mới	Người đến trước nhận WAITING; khi người thứ hai vào, cả hai nhận START	Đạt
7	Đánh khi chưa tới lượt; đánh vào ô đã có quân	Server bỏ qua, bàn cờ hai bên không đổi	Cả hai trường hợp server không phản hồi; nước hợp lệ (7,7) được gửi tới cả hai bên	Đạt
8	Để quá 20 giây không đánh	Server đánh thay một ô, ván không bị treo	Khoảng 19 đến 20 giây sau khi bắt đầu, server đánh thay một ô và gửi MOVE cho người chơi	Đạt
9	Xếp đủ 5 quân liên tiếp	Hai bên nhận thông báo thắng, có đường thắng	Cả hai nhận WIN, winnerId đúng, đường thắng gồm 5 ô	Đạt
10	Kiểm tra thống kê và rating sau ván thắng	Người thắng tăng điểm, người thua giảm điểm theo Elo	demo1 (1120) thắng nguyena (1250): 1120 → 1134 và 1250 → 1236, khớp công thức K = 20 (kỳ vọng 0,32 nên cộng 14 điểm)	Đạt

STT	Kịch bản	Kết quả mong đợi	Kết quả thực tế	Đánh giá
11	Một bên bấm đầu hàng	Bên còn lại được tính thắng	Bên còn lại nhận WIN với winnerId của mình	Đạt
12	Đề nghị hòa: từ chối, rồi đề nghị lại và đồng ý	Từ chối thì người đề nghị được báo; đồng ý thì hòa	Người đề nghị nhận DRAW_REJECT; lần sau cả hai nhận DRAW	Đạt
13	Ván kết thúc, một bên bấm "Chơi lại" rồi bên kia cũng bấm	Chỉ khi cả hai bấm mới bắt đầu ván mới	Một bên bấm: chưa có ván mới; cả hai bấm: cả hai nhận START	Đạt
14	Một bên tắt kết nối giữa ván	Bên kia nhận thông báo đối thủ rời đi	Bên còn lại nhận OPPONENT_LEFT	Đạt
15	Sau khi có người rớt mạng, đăng nhập mới	Server vẫn phục vụ bình thường	LOGIN_SUCCESS	Đạt
16	Xem bảng xếp hạng, lịch sử trận, đổi tên hiển thị	Có dữ liệu, xếp theo rating giảm dần	Nhận LEADERBOARD, MATCH_HISTORY (4 trận) và PROFILE_UPDATED	Đạt
17	Chơi với AI ở ba mức (máy đánh cả hai bên 60 nước)	Mọi nước đi đều hợp lệ (trong bàn, ô trống)	Cả ba mức đánh đủ 60 nước hợp lệ; thời gian cho 60 nước: Dễ 3 ms, Trung bình 14 ms, Khó 17 ms	Đạt
18	AI mức Trung bình và Khó khi có 4 quân liền hai đầu hở (của máy, rồi của người)	Máy đánh nước thắng; nước chặn	Cả hai mức đều chọn (7,4), một đầu của hàng 4 quân, trong cả hai tình huống	Đạt
19	Tắt MySQL giữa lúc đang đánh	Ván vẫn tiếp tục, lỗi chỉ hiện trong log	Các nước đi sau khi tắt MySQL vẫn được chuyển tiếp, ván kết thúc và hai bên nhận WIN; server in dòng "[GameRoom] Lỗi lưu nước đi vào DB"	Đạt

Bảng 3.7. Kịch bản kiểm thử và kết quả thực tế

Cả 19 kịch bản kiểm thử đều đạt yêu cầu. Riêng kịch bản 8 (hết giờ), thời gian kết thúc sớm hơn mốc 20 giây một chút do server bắt đầu đếm ngược ngay khi gửi thông điệp START, trong khi client cần thêm một khoảng thời gian để nhận được thông điệp này.

3.4.2. Kết quả chạy thử thực tế

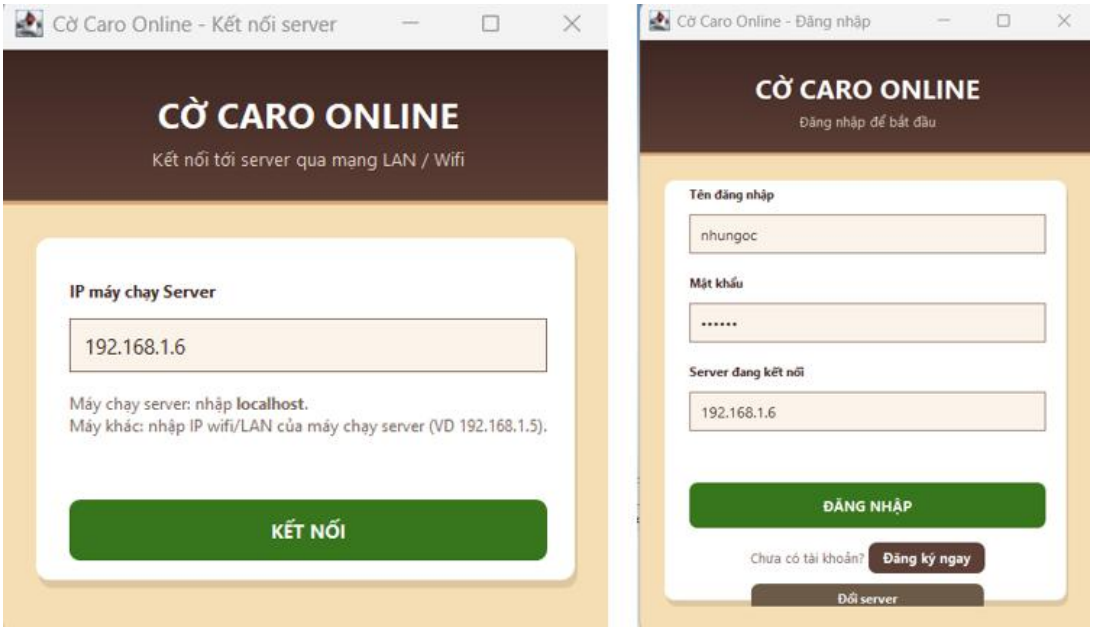
Đoạn nhật ký dưới đây được trích rút gọn từ cửa sổ server trong quá trình nhóm chạy thử chương trình trên máy phát triển; dấu ba chấm biểu thị những phần đã được lược bỏ. Nhật ký ghi nhận các yêu cầu đăng nhập và đăng ký được gửi đến AuthServer, các kết nối đến cổng chơi và thao tác đặt tên của người chơi. Ngoài ra, có thể thấy một ván đấu bắt đầu tại phòng ZPMBTA. Ở một trường hợp khác, người chơi tạo phòng LG7U37 nhưng sau đó rời đi, khiến phòng bị hủy.

```
Server đang lắng nghe tại port 12345 ...
[AuthServer] Đang lắng nghe tại port 12346 (xử lý đăng nhập/đăng ký)...
[AuthServer] Yêu cầu đăng nhập: nhungoc
[Server] Kết nối mới từ /192.168.1.6
[Server] Client kết nối: /192.168.1.6
[Server] như ngọc nè đặt tên (userId=4)
[AuthServer] Yêu cầu đăng ký: tham
[AuthServer] Yêu cầu đăng nhập: tham
[Server] Kết nối mới từ /192.168.1.6
[Server] Client kết nối: /192.168.1.6
[Server] ne đặt tên (userId=7)
[RoomManager] Bắt đầu ván tại phòng ZPMBTA: như ngọc nè vs ne
[Server] như ngọc nè đã ngắt kết nối.
[Server] Kết nối mới từ /192.168.1.6
[Server] Client kết nối: /192.168.1.6
[Server] như ngọc nè đặt tên (userId=4)
[Server] ne đã ngắt kết nối.
[Server] Kết nối mới từ /192.168.1.6
[Server] Client kết nối: /192.168.1.6
[Server] ne đặt tên (userId=7)
[RoomManager] như ngọc nè tạo phòng LG7U37
...
[Server] như ngọc nè đã ngắt kết nối.
[Server] như ngọc nè đã ngắt kết nối.
[RoomManager] Phòng LG7U37 đã bị hủy (host rời đi).
[Server] ne đã ngắt kết nối.
```

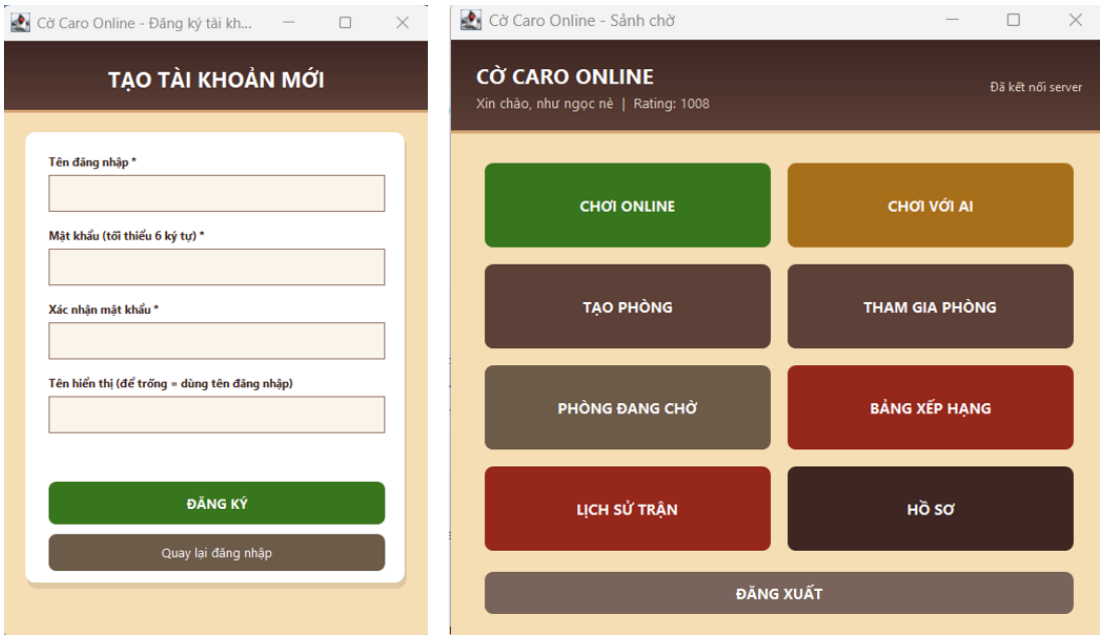
Mỗi sự kiện trong nhật ký server đều có tiền tố riêng như [Server], [AuthServer] và [RoomManager], giúp nhóm dễ theo dõi và xác định những gì đã xảy ra trong quá trình chạy chương trình. Cách ghi log này được áp dụng ngay từ đầu vì khi debug, nhật ký là một trong những nguồn thông tin quan trọng nhất để biết server đang xử lý những gì. Các cặp dòng "đã ngắt kết nối" và "Kết nối mới" xuất hiện liên tiếp thường là lúc người chơi quay về sảnh chờ. Khi đó, client đóng kết nối cũ rồi thiết lập một kết nối mới. Hiện tại, server chưa ghi log riêng cho từng nước đi hoặc kết quả ván đấu; những thông tin này được lưu trong hai bảng match_moves và matches. Riêng tiền tố [GameRoom] hiện chỉ xuất hiện khi xảy ra lỗi liên quan đến cơ sở dữ liệu.

Các hình tiếp theo là ảnh chụp trực tiếp từ chương trình trong quá trình chạy thử. Hình 3.4 minh họa màn hình nhập địa chỉ IP của server và màn hình đăng nhập. Hình

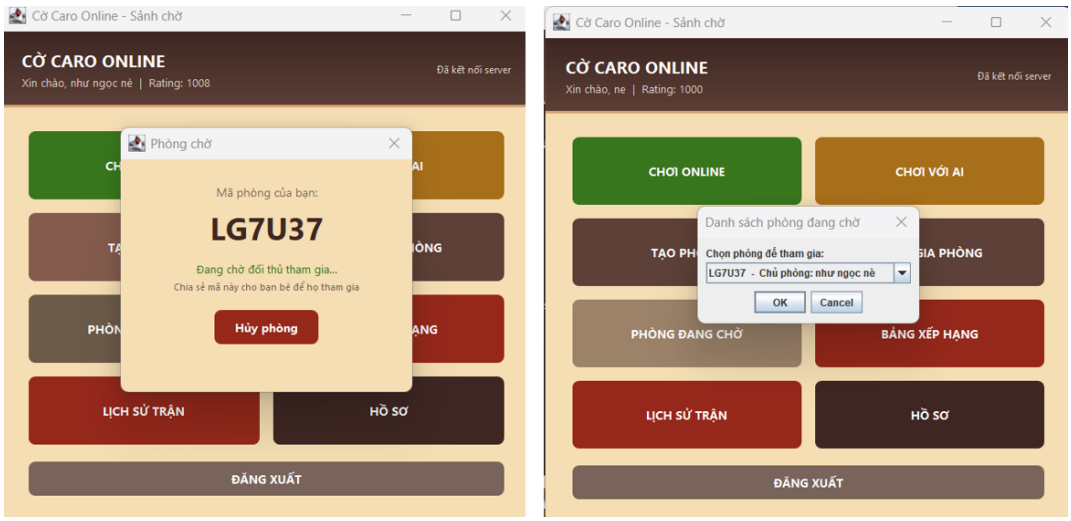
3.5 thể hiện giao diện đăng ký tài khoản và sảnh chờ, còn Hình 3.6 giới thiệu phòng chờ, bao gồm mã phòng và danh sách các phòng đang chờ người tham gia. Hình 3.7 ghi lại một ván đấu online giữa hai tài khoản: bên trái là máy thắng với năm quân X liên tiếp được tô xanh, bên phải là máy thua. Cả hai giao diện đều có khung chat cùng các nút đề nghị hòa, đầu hàng, chơi lại và trở về sảnh chờ. Tiếp theo, Hình 3.8 minh họa chế độ chơi với máy ở mức Khó; Hình 3.9 trình bày bảng xếp hạng và lịch sử trận đấu; cuối cùng, Hình 3.10 là cửa sổ server hiển thị nhật ký hoạt động.



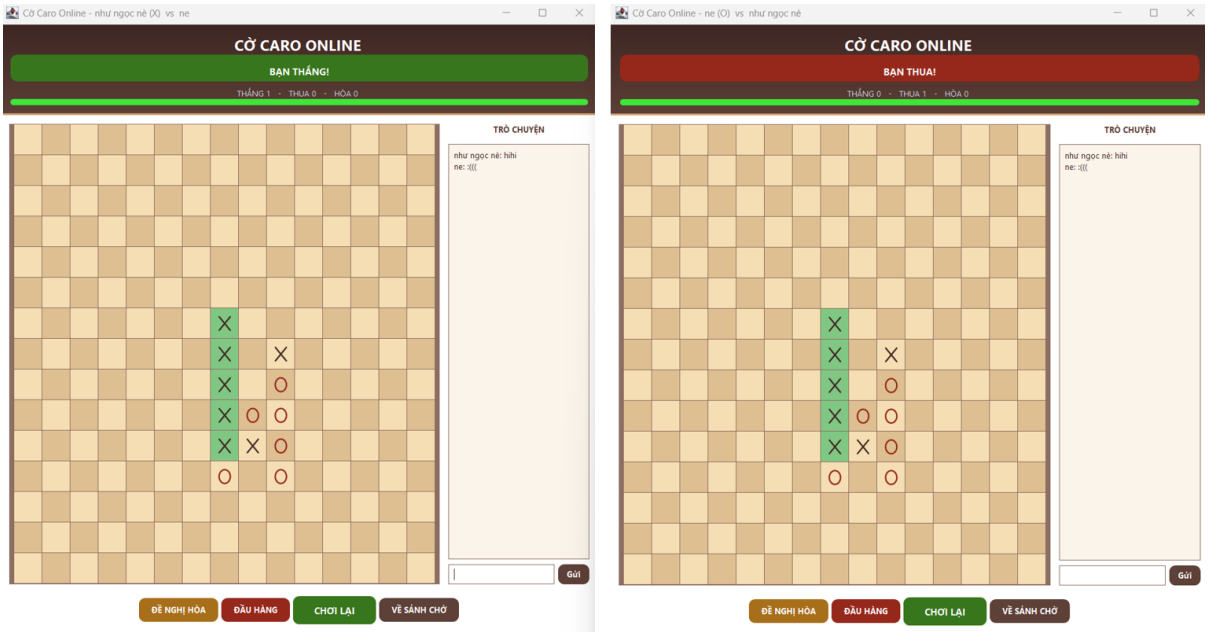
Hình 3.4. Màn hình kết nối server và đăng nhập



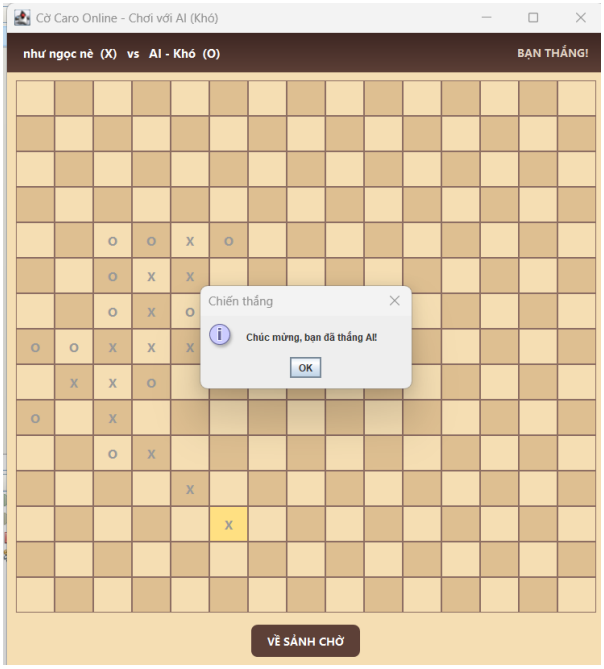
Hình 3.5. Màn hình đăng ký tài khoản và sảnh chờ



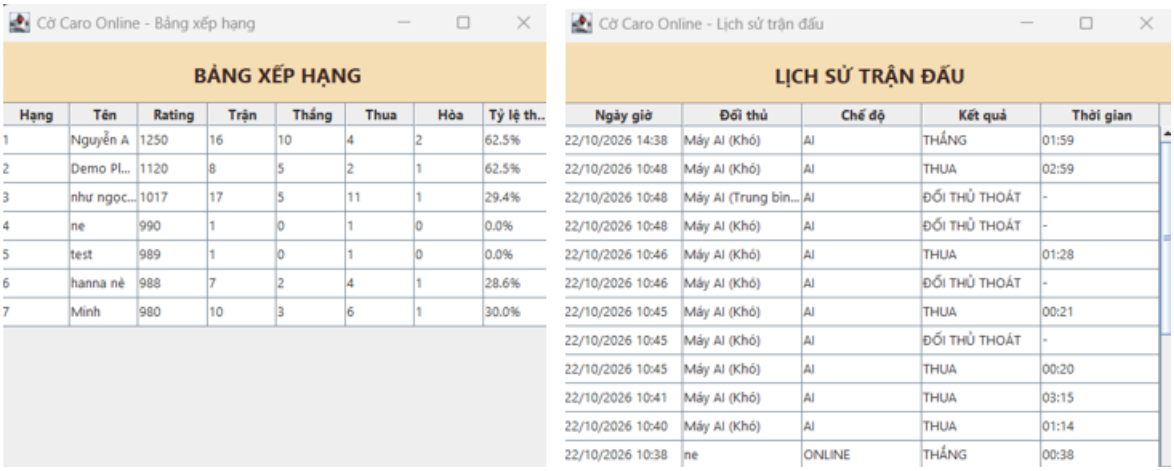
Hình 3.6. Phòng chờ và danh sách phòng đang chờ



Hình 3.7. Ván đấu online: bên thắng (trái) và bên thua (phải)



Hình 3.8. Giao diện chơi với máy

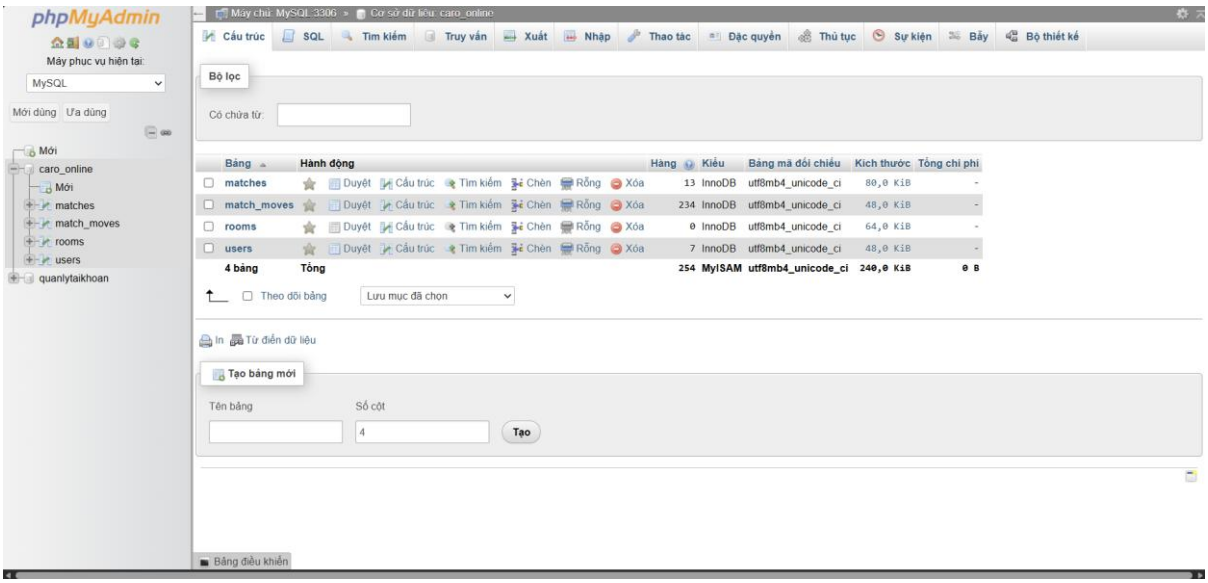


Hình 3.9. Bảng xếp hạng và lịch sử trận đấu

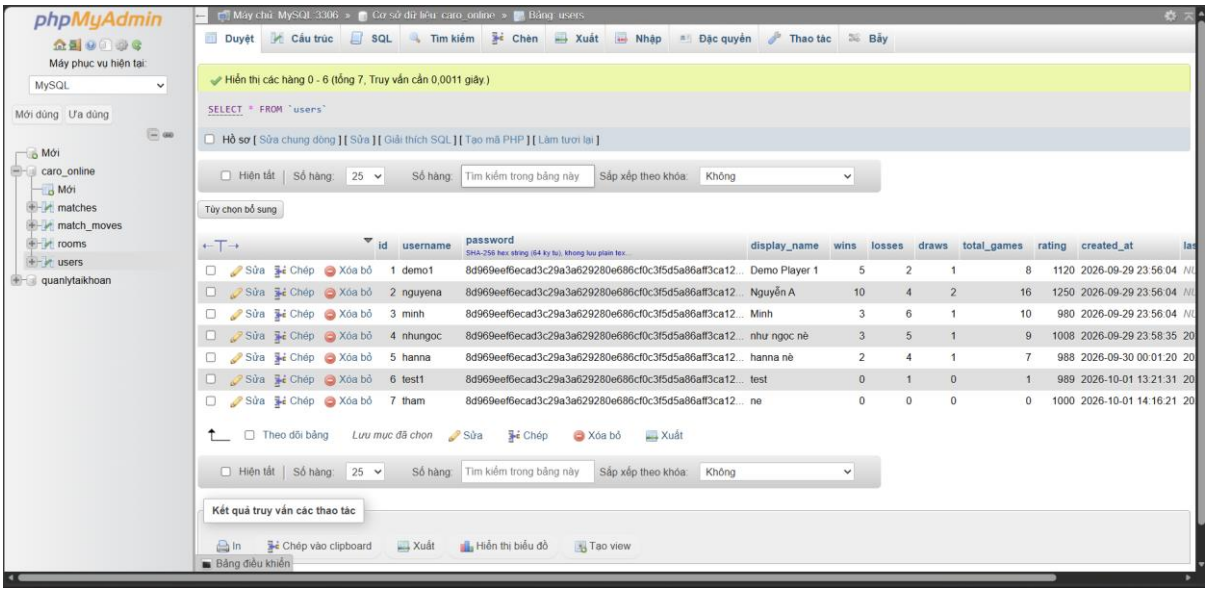


Hình 3.10. Cửa sổ server đang chạy

Sau khi hoàn tất các ván đấu, nhóm mở phpMyAdmin để kiểm tra dữ liệu được lưu trong cơ sở dữ liệu. Hình 3.11 cho thấy database caro_online có đầy đủ bốn bảng, trong đó matches có 13 dòng, match_moves có 234 dòng và users có 7 dòng. Riêng bảng rooms chưa có dữ liệu vì server hiện quản lý phòng trong bộ nhớ, chưa lưu thông tin vào bảng này. Hình 3.12 thể hiện dữ liệu trong bảng users. Cột password chỉ lưu chuỗi băm SHA-256 thay vì mật khẩu gốc, còn điểm rating và các thống kê thắng, thua, hòa được cập nhật sau mỗi ván đấu.



Hình 3.11. Các bảng của database caro_online trong phpMyAdmin



Hình 3.12. Dữ liệu bảng users trong phpMyAdmin

Trong quá trình phát triển, nhóm gặp một lỗi khá khó tìm ra nguyên nhân. Khi vừa kết nối, client gửi thông điệp NAME để thông báo tên người chơi. Tuy nhiên, lúc này client chưa thuộc phòng nào nên server không thể chuyển tiếp tên đó cho đối thủ, khiến hai người chơi không nhìn thấy tên của nhau. Để khắc phục, nhóm điều chỉnh RoomManager để chủ động gửi tên của cả hai người chơi cho nhau ngay khi ghép phòng thành công.

3.4.3. Đánh giá ưu điểm và hạn chế

Ưu điểm: Server đóng vai trò trọng tài duy nhất, giúp trạng thái bàn cờ trên hai client luôn đồng bộ và ngăn client tự quyết định kết quả thắng thua. Các phòng hoạt động độc lập nên nhiều ván đấu có thể diễn ra đồng thời mà không ảnh hưởng lẫn nhau. Việc tách riêng hai cổng kết nối cũng giúp phân biệt chức năng đăng nhập với xử lý trò chơi. Ngoài ra, lỗi ghi cơ sở dữ liệu được bắt và ghi log, hạn chế ảnh hưởng đến ván đấu

đang diễn ra. Với chế độ chơi cùng AI, chương trình xử lý trực tiếp trên client nên không cần kết nối đến server.

Hạn chế: Bên cạnh những chức năng đã hoàn thành, hệ thống vẫn còn một số điểm cần cải thiện.

Thứ nhất, mật khẩu được client gửi đến server dưới dạng văn bản thuần qua socket, sau đó server mới băm SHA-256 để so sánh. Dữ liệu truyền chưa được mã hóa và giá trị băm chưa có salt, nên cơ chế này chưa đảm bảo an toàn nếu triển khai trên Internet. Với mục đích học tập trong mạng LAN, cách làm hiện tại giúp đơn giản hóa việc xây dựng hệ thống, nhưng cần được nâng cấp trước khi sử dụng thực tế.

Thứ hai, hệ thống chưa có chức năng phân quyền quản trị. Tất cả tài khoản hiện đều có quyền như nhau.

Thứ ba, mỗi khi chọn Về sảnh chờ, client phải đóng kết nối hiện tại rồi tạo kết nối mới đến server. Nếu thao tác này diễn ra khi ván đấu chưa kết thúc, hệ thống sẽ xem người chơi là đã rời trận và gửi thông báo OPPONENT_LEFT cho đối thủ.

Thứ tư, chế độ AI hiện cho phép client tự ghi kết quả ván đấu vào cơ sở dữ liệu, nên người dùng về lý thuyết có thể can thiệp vào dữ liệu. Rating sau mỗi ván đấu với máy được tính dựa trên mức rating giả định cố định cho từng độ khó: Dễ là 800, Trung bình là 1000 và Khó là 1300. Ngoài ra, chế độ này chưa hỗ trợ đồng hồ đếm ngược và khung chat.

Thứ năm, cơ chế ghép trận ngẫu nhiên chỉ dựa vào thứ tự người chơi tham gia, chưa xét đến rating. Vì vậy, người mới chơi có thể phải đấu với người có trình độ cao hơn đáng kể.

Thứ sáu, khi hết thời gian 20 giây, server tự động đánh một ô ngẫu nhiên. Cách xử lý này giúp ván đấu tiếp tục nhưng đôi khi khiến người chơi rơi vào thế bất lợi, dù họ chỉ đang suy nghĩ nước đi.

Thứ bảy, thông tin kết nối như địa chỉ cơ sở dữ liệu, tài khoản MySQL và số cổng hiện được khai báo trực tiếp trong mã nguồn, chưa được tách ra file cấu hình. Điều này khiến việc thay đổi môi trường triển khai kém linh hoạt.

Thứ tám, hệ thống sử dụng Java Serialization để trao đổi dữ liệu nên client phải được viết bằng Java hoặc tương thích với định dạng này. Bên cạnh đó, mô hình mỗi client sử dụng một luồng riêng có thể gây khó khăn khi mở rộng lên số lượng kết nối lớn, chẳng hạn hàng nghìn người chơi đồng thời.

Cuối cùng, bộ kịch bản kiểm thử trong mục 3.4 được xây dựng bằng chương trình riêng trong thư mục KiemThu, chưa tích hợp trực tiếp vào project chính. Giao diện Swing cũng chưa có bộ kiểm thử tự động, nên việc kiểm tra giao diện vẫn chủ yếu dựa vào thao tác thủ công.

CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ

Đồ án đã hoàn thành việc xây dựng hệ thống cờ Caro trực tuyến hoạt động trong mạng LAN, đáp ứng các mục tiêu đề ra ở Chương 1. Hệ thống hỗ trợ người chơi đăng ký, đăng nhập, tạo phòng, ghép trận và thi đấu với nhau, với thời gian 20 giây cho mỗi lượt đi. Ngoài ra, người chơi có thể đầu hàng, đề nghị hòa, xem lại lịch sử thi đấu và theo dõi thứ hạng trên bảng xếp hạng. Trong trường hợp không có đối thủ, người chơi cũng có thể đấu với máy ở ba mức độ khó khác nhau. Kết quả các ván đấu được lưu trữ trong cơ sở dữ liệu MySQL.

Trong quá trình thực hiện, một trong những kinh nghiệm quan trọng nhất của nhóm là xây dựng server làm trọng tài chính cho mọi ván đấu. Cách thiết kế này giúp việc đồng bộ bàn cờ giữa hai người chơi trở nên đáng tin cậy hơn, đồng thời hỗ trợ kiểm soát luật chơi và xử lý các trường hợp mất kết nối. Bên cạnh đó, quá trình triển khai đa luồng giúp nhóm hiểu rõ hơn về cách xử lý các yêu cầu đồng thời, cũng như cách sử dụng synchronized và ConcurrentHashMap để hạn chế những vấn đề phát sinh khi nhiều người chơi hoạt động cùng lúc.

Tuy nhiên, hệ thống vẫn còn một số hạn chế đã được trình bày tại mục 3.4.3. Nếu có thêm thời gian phát triển, nhóm đề xuất một số hướng cải tiến như sau:

Bảo mật: Mã hóa dữ liệu truyền qua mạng bằng TLS và sử dụng thuật toán băm mật khẩu có salt, chẳng hạn như bcrypt, nhằm tăng cường khả năng bảo vệ tài khoản.

Trải nghiệm người dùng: Cải thiện cơ chế ghép trận bằng cách cân nhắc điểm rating của người chơi thay vì chỉ dựa vào thứ tự tham gia. Đồng thời, duy trì kết nối khi người chơi quay lại sảnh để tránh phải thiết lập kết nối mới.

Xem lại ván đấu: Tận dụng dữ liệu nước đi đã được lưu trong bảng match_moves để xây dựng chức năng phát lại các ván đấu đã hoàn thành.

Trí tuệ nhân tạo: Nâng cấp HardAI bằng thuật toán Minimax kết hợp cắt tỉa Alpha-Beta để cải thiện khả năng tính toán và lựa chọn nước đi của máy.

Kiến trúc hệ thống: Sử dụng thread pool thay cho việc tạo một luồng riêng cho mỗi client, đồng thời chuyển giao thức trao đổi dữ liệu sang JSON để thuận tiện hơn khi phát triển client trên các nền tảng khác nhau.

Vận hành và kiểm thử: Đưa các thông số kết nối vào file cấu hình, bổ sung chức năng quản trị và xây dựng các bài kiểm thử tự động cho những luồng quan trọng như ghép phòng, xử lý nước đi và xác định kết quả ván đấu.

Nhìn chung, đồ án là cơ hội để nhóm vận dụng kiến thức đã học vào việc xây dựng một hệ thống mạng có nhiều người dùng hoạt động đồng thời. Trong quá trình thực hiện, nhóm không chỉ củng cố kiến thức về lập trình Java, cơ sở dữ liệu và xử lý đa luồng mà còn tích lũy được kinh nghiệm thực tế khi kiểm thử, phát hiện và khắc phục lỗi trên nhiều máy tính. Đây sẽ là nền tảng hữu ích để nhóm tiếp tục hoàn thiện kỹ năng và phát triển những hệ thống có quy mô lớn hơn trong tương lai.

TÀI LIỆU THAM KHẢO

[1] Oracle. Java Tutorials – All About Sockets. Truy cập tháng 10/2026. https://docs.oracle.com/javase/tutorial/networking/sockets/index.html

[2] Oracle. Java SE 8 API – ObjectOutputStream. Truy cập tháng 10/2026. https://docs.oracle.com/javase/8/docs/api/java/io/ObjectOutputStream.html

[3] Oracle. Java Tutorials – Concurrency. Truy cập tháng 10/2026. https://docs.oracle.com/javase/tutorial/essential/concurrency/

[4] Oracle. Java Tutorials – Creating a GUI With Swing. Truy cập tháng 10/2026. https://docs.oracle.com/javase/tutorial/uiswing/

[5] Oracle. MySQL Connector/J Developer Guide. Truy cập tháng 10/2026. https://dev.mysql.com/doc/connector-j/en/

[6] Postel, J. (1981). RFC 793 – Transmission Control Protocol. https://www.rfc-editor.org/rfc/rfc793

[7] Wikipedia. Elo rating system. Truy cập tháng 10/2026. https://en.wikipedia.org/wiki/Elo_rating_system

[8] Wikipedia. Gomoku. Truy cập tháng 10/2026. https://en.wikipedia.org/wiki/Gomoku

[9] Nhóm 9. Caro Online (mã nguồn chương trình). GitHub. Truy cập tháng 10/2026. https://github.com/nhungoc260/caro-online-java

PHỤ LỤC: MÃ NGUỒN CHƯƠNG TRÌNH

Mã nguồn đầy đủ nằm trong thư mục src/caro của project CaroOnline (35 file Java chia làm 6 package) và file database/caro_online.sql. Toàn bộ project cũng được đăng tại https://github.com/nhungoc260/caro-online-java. Phụ lục này liệt kê toàn bộ nội dung các file chính: Message.java, CaroServer.java, ClientHandler.java, RoomManager.java, GameRoom.java, AuthServer.java, AuthService.java, UserDAO.java, MediumAI.java và HardAI.java.

B.1. common/Message.java

```
package caro.common;

import java.io.Serializable;

/**
 * Lớp Message dùng để đóng gói dữ liệu trao đổi giữa Client và Server
 * qua ObjectOutputStream / ObjectInputStream (Serialization).
 *
 * Class này PHẢI giống hệt nhau ở cả phía server và client,
 * nếu không sẽ bị lỗi ClassNotFoundException / InvalidClassException.
 */
public class Message implements Serializable {

    private static final long serialVersionUID = 1L;

    public enum Type {
        NAME,           // Client -> Server: gửi tên người chơi ngay sau khi kết
nối
                        // Server -> Client: chuyển tiếp tên của đối thủ
        WAITING,        // Server -> Client: đang chờ đối thủ vào phòng
        START,          // Server -> Client: đủ 2 người, bắt đầu ván mới
        MOVE,           // Client -> Server: người chơi đánh 1 nước
                        // Server -> Client: broadcast lại nước đi cho cả 2 bên
        WIN,            // Server -> Client: có người thắng
        DRAW,           // Server -> Client: hòa (hết ô trống)
        OPPONENT_LEFT,  // Server -> Client: đối thủ đã ngắt kết nối
        REPLAY,         // Client -> Server: xin chơi lại
                        // Server -> Client: báo đối thủ đang chờ chơi lại
        CHAT,           // Client -> Server: gửi tin nhắn chat
                        // Server -> Client: chuyển tiếp tin nhắn (kèm tên người
gửi)
        ERROR,

        // ==== PHASE 2: Đăng ký / Đăng nhập (đi qua AuthServer, cổng riêng) ====
        LOGIN,              // Client -> Server: gửi username + password để đăng
nhập
        LOGIN_SUCCESS,      // Server -> Client: đăng nhập đúng, kèm theo "user"
(thông tin tài khoản)
        LOGIN_FAILED,       // Server -> Client: sai username/password, lý do nằm
trong "note"
        REGISTER,           // Client -> Server: gửi username + password +
displayName để đăng ký
        REGISTER_SUCCESS,   // Server -> Client: đăng ký thành công, kèm theo
"user"
        REGISTER_FAILED,    // Server -> Client: đăng ký thất bại, lý do nằm trong
"note"

        // ==== PHASE 4/5: Lobby + Room + Matchmaking ====
```

```
        CREATE_ROOM,       // Client -> Server: tạo phòng mới
        ROOM_CREATED,      // Server -> Client: tạo phòng thành công, roomCode
chứa mã phòng
        JOIN_ROOM,         // Client -> Server: xin vào phòng, roomCode chứa mã
phòng muốn vào
        ROOM_JOINED,       // Server -> Client (cả 2 bên): vào phòng thành công
        ROOM_JOIN_FAILED,  // Server -> Client: không vào được phòng, lý do trong
"note"
        LEAVE_ROOM,        // Client -> Server: rời phòng đang chờ (trước khi đủ 2
người)
        ROOM_LIST,         // Client -> Server: xin danh sách phòng đang chờ
                           // Server -> Client: trả về danh sách, chứa trong
"roomList"
        FIND_MATCH,        // Client -> Server: xin ghép trận ngẫu nhiên
(matchmaking)
        CANCEL_FIND,       // Client -> Server: hủy tìm trận

        // ==== PHASE 7: Đầu hàng / Đề nghị hòa ====
        SURRENDER,         // Client -> Server: đầu hàng
        OFFER_DRAW,        // Client -> Server: đề nghị hòa
                           // Server -> Client: chuyển tiếp đề nghị hòa cho đối
thủ
        DRAW_ACCEPT,       // Client -> Server: đồng ý đề nghị hòa
        DRAW_REJECT,       // Client -> Server: từ chối đề nghị hòa
                           // Server -> Client: báo người đề nghị biết đã bị từ
chối

        // ==== PHASE 8/9: Lịch sử + Bảng xếp hạng ====
        MATCH_HISTORY,     // Client -> Server: xin lịch sử trận đấu của mình
                           // Server -> Client: trả về, chứa trong "matchHistory"
        LEADERBOARD,       // Client -> Server: xin bảng xếp hạng
                           // Server -> Client: trả về, chứa trong "leaderboard"

        // ==== Phase 4: Hồ sơ ====
        PROFILE_UPDATE,    // Client -> Server: đổi tên hiển thị, displayName chứa
tên mới
        PROFILE_UPDATED    // Server -> Client: cập nhật thành công, kèm theo
"user" mới
    }

    public Type type;
    public int playerId;    // 1 hoặc 2 - id người chơi
    public int x, y;        // toạ độ ô cờ (dùng cho MOVE)
    public int nextTurn;    // playerId của người được đánh tiếp theo (dùng cho
MOVE)
    public int winnerId;    // playerId người thắng (dùng cho WIN)
    public String note;     // tên người chơi / thông điệp phụ / thông báo lỗi,
hiển thị lên GUI
    public int[] winLineX;  // toạ độ x của các ô nằm trên đường thắng (để tô
sáng)
    public int[] winLineY;  // toạ độ y của các ô nằm trên đường thắng (để tô
sáng)

    // ==== PHASE 2: các field phục vụ LOGIN/REGISTER ====
    public String username;      // dùng cho LOGIN, REGISTER
    public String password;      // dùng cho LOGIN, REGISTER (dạng plain text
TRONG BỘ NHỚ,
                                 // server sẽ hash trước khi so sánh/lưu - không
bao giờ lưu plain text)
    public String displayName;   // dùng cho REGISTER, PROFILE_UPDATE
```

```
    public User user;            // dùng cho LOGIN_SUCCESS, REGISTER_SUCCESS,
PROFILE_UPDATED

    // ==== PHASE 4/5: Lobby/Room/Matchmaking ====
    public String roomCode;                 // dùng cho
CREATE_ROOM/ROOM_CREATED/JOIN_ROOM/ROOM_JOINED
    public java.util.List<RoomInfo> roomList;       // dùng cho ROOM_LIST
(response)

    // ==== PHASE 8/9: Lịch sử + Bảng xếp hạng ====
    public java.util.List<MatchSummary> matchHistory; // dùng cho MATCH_HISTORY
(response)
    public java.util.List<User> leaderboard;           // dùng cho LEADERBOARD
(response)

    public Message(Type type) {
        this.type = type;
    }
}
```

B.2. server/CaroServer.java

```
package caro.server;

import java.io.IOException;
import java.net.BindException;
import java.net.InetAddress;
import java.net.NetworkInterface;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketException;
import java.util.Collections;
import java.util.Enumeration;

/**
 * Main class của Server.
 * Chạy file này TRƯỚC, sau đó mới chạy CaroClient (2 lần) để vào chơi.
 */
public class CaroServer {

    public static final int PORT = 12345;

    public static void main(String[] args) {
        System.out.println("=== CARO SERVER ===");
        printLocalIpAddresses();

        // PHASE 2: khởi động AuthServer (cổng riêng 12346, xử lý đăng nhập/đăng
ký)
        // chạy nền song song - KHÔNG ảnh hưởng gì tới vòng lặp ghép game bên
dưới.
        AuthServer.startInBackground();

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server đang lắng nghe tại port " + PORT + " ...");
            System.out.println("(PHASE 5: mỗi kết nối được xử lý ngay bởi 1 thread
riêng,");
            System.out.println(" việc ghép cặp 2 người chơi do RoomManager quyết
định qua Lobby)\n");

            while (true) {
                Socket socket = serverSocket.accept();
                ClientHandler handler = new ClientHandler(socket);
                new Thread(handler).start();
```

```
                System.out.println("[Server] Kết nối mới từ " +
socket.getInetAddress());
            }
        } catch (BindException e) {
            System.out.println();
            System.out.println("!!! KHÔNG THỂ KHỞI ĐỘNG SERVER !!!");
            System.out.println("Port " + PORT + " đang bị 1 tiến trình Server khác
chiếm giữ");
            System.out.println("(có thể bạn đã Run File CaroServer.java từ trước
đó và quên tắt).");
            System.out.println();
            System.out.println("Cách khắc phục:");
            System.out.println("  1. Nhìn xuống khung Output, tìm tab Server cũ
đang chạy,");
            System.out.println("     bấm nút vuông đỏ (Stop) để tắt nó đi.");
            System.out.println("  2. Nếu không tìm thấy, đóng và mở lại NetBeans
rồi thử lại.");
            System.out.println();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * In ra các địa chỉ IP nội bộ (LAN) của máy đang chạy server,
     * để dễ đọc và nhập vào Client khi demo qua LAN/hotspot thật,
     * khỏi phải tự gõ lệnh ipconfig / ifconfig.
     */
    private static void printLocalIpAddresses() {
        try {
            System.out.println("Địa chỉ IP của máy này (dùng IP nào để bạn cùng
team nhập vào Client):");
            Enumeration<NetworkInterface> interfaces =
NetworkInterface.getNetworkInterfaces();
            for (NetworkInterface ni : Collections.list(interfaces)) {
                if (!ni.isUp() || ni.isLoopback()) continue;
                Enumeration<InetAddress> addresses = ni.getInetAddresses();
                for (InetAddress addr : Collections.list(addresses)) {
                    // Chỉ in IPv4 cho dễ đọc (VD: 192.168.x.x)
                    if (addr.getHostAddress().indexOf(':') == -1) {
                        System.out.println("   - " + ni.getDisplayName() + " : " +
addr.getHostAddress());
                    }
                }
            }
        } catch (SocketException e) {
            System.out.println("   (Không lấy được danh sách IP, dùng lệnh
ipconfig/ifconfig để xem thủ công)");
        }
        System.out.println();
    }
}
```

B.3. server/ClientHandler.java

```
package caro.server;

import caro.common.MatchSummary;
import caro.common.Message;
import caro.common.User;
import caro.database.DatabaseConnection;
import caro.database.MatchDAO;
import caro.database.UserDAO;
```

```
import java.io.EOFException;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.net.Socket;
import java.net.SocketException;
import java.util.ArrayList;
import java.util.List;

/**
 * Mỗi client kết nối vào server sẽ được gán 1 ClientHandler,
 * chạy trên 1 Thread riêng (đáp ứng yêu cầu "TCP + Thread").
 *
 * PHASE 4/5: Khác với bản gốc (chỉ xử lý MOVE/REPLAY/CHAT trong 1 GameRoom
 * cố định), ClientHandler giờ xử lý CẢ giai đoạn "ở Lobby" (tạo/tham gia
 * phòng, matchmaking, xem bảng xếp hạng/lịch sử) LẪN giai đoạn "đang chơi"
 * (room != null) - tất cả trên CÙNG 1 kết nối, CÙNG 1 vòng lặp đọc Message,
 * y hệt tinh thần code gốc, chỉ mở rộng thêm case trong switch.
 */
public class ClientHandler implements Runnable {

    private final Socket socket;
    private ObjectOutputStream out;
    private ObjectInputStream in;
    private int playerId;
    private GameRoom room;
    private String playerName = "Người chơi";

    // ==== PHASE 4: thông tin tài khoản đã đăng nhập (null nếu là kết nối
    // "vô danh" chạy CaroClient.java trực tiếp không qua LoginFrame) ====
    private Integer userId;
    private int rating = 1000;

    // ==== PHASE 5: mã phòng hiện đang đứng chờ (chỉ có giá trị khi ở trạng
    // thái WAITING trong 1 phòng do chính mình tạo) ====
    private String currentRoomCode;

    private final UserDAO userDAO = new UserDAO();
    private final MatchDAO matchDAO = new MatchDAO();

    public ClientHandler(Socket socket) {
        this.socket = socket;
    }

    public void setPlayerId(int id) {
        this.playerId = id;
        // Chỉ đặt tên mặc định "Người chơi N" nếu CHƯA có tên thật nào được
        // set từ trước (qua NAME/đăng nhập) - tránh việc gán playerId lúc
        // ghép phòng (Phase 5) đè mất tên thật đã có.
        if (this.playerName == null || this.playerName.equals("Người chơi")) {
            this.playerName = "Người chơi " + id;
        }
    }

    public void setRoom(GameRoom room) { this.room = room; }
    public String getPlayerName() { return playerName; }
    public Integer getUserId() { return userId; }
    public int getRating() { return rating; }
    public void setCurrentRoomCode(String code) { this.currentRoomCode = code; }
    public String getCurrentRoomCode() { return currentRoomCode; }
```

```
    @Override
    public void run() {
        try {
            out = new ObjectOutputStream(socket.getOutputStream());
            out.flush();
            in = new ObjectInputStream(socket.getInputStream());

            System.out.println("[Server] Client kết nối: " +
socket.getInetAddress());

            Message msg;
            while ((msg = (Message) in.readObject()) != null) {
                handle(msg);
            }
        } catch (EOFException | SocketException e) {
            System.out.println("[Server] " + playerName + " đã ngắt kết nối.");
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            if (room != null) room.notifyOpponentLeft(this);
            RoomManager.INSTANCE.handleDisconnect(this);
            if (userId != null) {
                try {
                    userDAO.updateStatus(userId, "OFFLINE");
                } catch (DatabaseConnection.DatabaseException ignored) {
                }
            }
            try {
                socket.close();
            } catch (IOException ignored) {
            }
        }
    }

    private void handle(Message msg) {
        switch (msg.type) {
            case NAME:
                if (msg.note != null && !msg.note.trim().isEmpty()) {
                    playerName = msg.note.trim();
                }
                if (msg.user != null) {
                    userId = msg.user.getId();
                    rating = msg.user.getRating();
                }
                System.out.println("[Server] " + playerName + " đặt tên"
                        + (userId != null ? " (userId=" + userId + ")" : " (vô
danh)"));
                if (room != null) room.notifyNameUpdated(this);
                break;

            // ---- Lobby: tạo/tham gia/rời phòng, danh sách phòng, matchmaking --
--
            case CREATE_ROOM:
                RoomManager.INSTANCE.createRoom(this);
                break;

            case JOIN_ROOM:
                RoomManager.INSTANCE.joinRoom(this, msg.roomCode);
                break;
```

```
            case LEAVE_ROOM:
                RoomManager.INSTANCE.leaveRoom(this);
                break;

            case ROOM_LIST:
                sendMessage(RoomManager.INSTANCE.buildRoomListMessage());
                break;

            case FIND_MATCH:
                RoomManager.INSTANCE.enqueueMatchmaking(this);
                break;

            case CANCEL_FIND:
                RoomManager.INSTANCE.cancelMatchmaking(this);
                break;

            // ---- Trong ván đấu ----
            case MOVE:
                if (room != null) room.handleMove(playerId, msg.x, msg.y);
                break;

            case REPLAY:
                if (room != null) room.requestReplay(playerId);
                break;

            case CHAT:
                if (room != null) room.broadcastChat(this, msg.note);
                break;

            case SURRENDER:
                if (room != null) room.surrender(playerId);
                break;

            case OFFER_DRAW:
                if (room != null) room.offerDraw(playerId);
                break;

            case DRAW_ACCEPT:
                if (room != null) room.respondDraw(playerId, true);
                break;

            case DRAW_REJECT:
                if (room != null) room.respondDraw(playerId, false);
                break;

            // ---- Bảng xếp hạng / Lịch sử / Hồ sơ ----
            case LEADERBOARD:
                sendLeaderboard();
                break;

            case MATCH_HISTORY:
                sendMatchHistory();
                break;

            case PROFILE_UPDATE:
                handleProfileUpdate(msg);
                break;

            default:
                break;
        }
```

```
    }

    private void sendLeaderboard() {
        try {
            List<User> top = userDAO.getLeaderboard(50);
            Message reply = new Message(Message.Type.LEADERBOARD);
            reply.leaderboard = top;
            sendMessage(reply);
        } catch (DatabaseConnection.DatabaseException e) {
            Message err = new Message(Message.Type.ERROR);
            err.note = "Không thể tải bảng xếp hạng: " + e.getMessage();
            sendMessage(err);
        }
    }

    private void sendMatchHistory() {
        if (userId == null) {
            Message err = new Message(Message.Type.ERROR);
            err.note = "Bạn cần đăng nhập để xem lịch sử trận đấu.";
            sendMessage(err);
            return;
        }
        try {
            List<MatchDAO.MatchHistoryItem> items =
matchDAO.getHistoryForUser(userId, 100);
            List<MatchSummary> summaries = new ArrayList<MatchSummary>();
            for (MatchDAO.MatchHistoryItem item : items) {
                MatchSummary s = new MatchSummary();
                s.matchId = item.matchId;
                s.opponentName = item.opponentName;
                s.mode = item.mode;
                s.result = item.result;
                s.startedAt = item.startedAt;
                s.durationSeconds = item.durationSeconds;
                summaries.add(s);
            }
            Message reply = new Message(Message.Type.MATCH_HISTORY);
            reply.matchHistory = summaries;
            sendMessage(reply);
        } catch (DatabaseConnection.DatabaseException e) {
            Message err = new Message(Message.Type.ERROR);
            err.note = "Không thể tải lịch sử trận đấu: " + e.getMessage();
            sendMessage(err);
        }
    }

    private void handleProfileUpdate(Message msg) {
        if (userId == null || msg.displayName == null ||
msg.displayName.trim().isEmpty()) {
            Message err = new Message(Message.Type.ERROR);
            err.note = "Không thể cập nhật hồ sơ.";
            sendMessage(err);
            return;
        }
        try {
            userDAO.updateDisplayName(userId, msg.displayName.trim());
            User updated = userDAO.getById(userId);
            playerName = updated.getDisplayName();
            Message reply = new Message(Message.Type.PROFILE_UPDATED);
            reply.user = updated;
            sendMessage(reply);
```

```
        } catch (DatabaseConnection.DatabaseException e) {
            Message err = new Message(Message.Type.ERROR);
            err.note = "Lỗi cập nhật hồ sơ: " + e.getMessage();
            sendMessage(err);
        }
    }

    /**
     * Gửi message xuống client của người chơi này.
     * synchronized để tránh 2 luồng ghi cùng lúc lên 1 stream.
     */
    public synchronized void sendMessage(Message m) {
        try {
            out.writeObject(m);
            out.flush();
            out.reset(); // tránh ObjectOutputStream cache lại object cũ khi gửi
nhiều lần
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

B.4. server/RoomManager.java

```
package caro.server;

import caro.common.Message;
import caro.common.RoomInfo;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Quản lý TẤT CẢ các phòng chơi (multi-room) và hàng đợi "Tìm đối thủ"
 * (matchmaking). Đây là điểm khác biệt lớn nhất so với server cũ: thay vì
 * chỉ ghép cố định 2 client 1 lần rồi lặp lại, giờ server có thể xử lý
 * NHIỀU phòng cùng lúc.
 *
 * GameRoom.java (logic bàn cờ/thắng-thua/timer) được TÁI SỬ DỤNG NGUYÊN VẸN
 * - class này chỉ lo việc "ai chơi với ai", không đụng gì tới luật chơi.
 */
public class RoomManager {

    /** Instance dùng chung cho toàn bộ server (1 server = 1 RoomManager). */
    public static final RoomManager INSTANCE = new RoomManager();

    private final ConcurrentHashMap<String, RoomEntry> rooms = new
ConcurrentHashMap<String, RoomEntry>();
    private final Random random = new Random();

    /** Người chơi đang chờ được ghép ngẫu nhiên (FIND_MATCH), tối đa 1 người tại
1 thời điểm. */
    private ClientHandler waitingForMatch = null;
    private final Object matchLock = new Object();

    private RoomManager() {
    }

    /** 1 phòng = mã phòng + GameRoom (logic) + host/guest + trạng thái hiển thị ở
Lobby. */
```

```
    private static class RoomEntry {
        String roomCode;
        GameRoom gameRoom;
        ClientHandler host;
        ClientHandler guest;
        String status; // WAITING / PLAYING
    }

    // ------------------------------------------------------
    // TẠO PHÒNG / THAM GIA PHÒNG (theo mã)
    // ------------------------------------------------------

    public synchronized void createRoom(ClientHandler host) {
        String code = generateUniqueRoomCode();
        RoomEntry entry = new RoomEntry();
        entry.roomCode = code;
        entry.gameRoom = new GameRoom();
        entry.gameRoom.setRoomCode(code);
        entry.host = host;
        entry.status = "WAITING";
        rooms.put(code, entry);

        host.setCurrentRoomCode(code);

        Message reply = new Message(Message.Type.ROOM_CREATED);
        reply.roomCode = code;
        host.sendMessage(reply);
        System.out.println("[RoomManager] " + host.getPlayerName() + " tạo phòng "
+ code);
    }

    public synchronized void joinRoom(ClientHandler guest, String roomCode) {
        RoomEntry entry = (roomCode == null) ? null :
rooms.get(roomCode.trim().toUpperCase());
        if (entry == null) {
            fail(guest, "Không tìm thấy phòng \"" + roomCode + "\".");
            return;
        }
        if (!"WAITING".equals(entry.status)) {
            fail(guest, "Phòng \"" + roomCode + "\" đã đầy hoặc đang chơi.");
            return;
        }
        if (entry.host == guest) {
            fail(guest, "Bạn không thể tự tham gia phòng của chính mình.");
            return;
        }

        entry.guest = guest;
        entry.status = "PLAYING";
        guest.setCurrentRoomCode(entry.roomCode);
        startGameInRoom(entry);
    }

    /** Rời phòng đang WAITING (trước khi có guest) - ví dụ host đổi ý không muốn
chờ nữa. */
    public synchronized void leaveRoom(ClientHandler who) {
        String code = who.getCurrentRoomCode();
        if (code == null) return;
        RoomEntry entry = rooms.get(code);
        if (entry != null && "WAITING".equals(entry.status) && entry.host == who)
{
```

```
            rooms.remove(code);
            System.out.println("[RoomManager] Phòng " + code + " đã bị hủy (host
rời đi).");
        }
        who.setCurrentRoomCode(null);
    }

    public synchronized Message buildRoomListMessage() {
        List<RoomInfo> list = new ArrayList<RoomInfo>();
        for (RoomEntry e : rooms.values()) {
            if ("WAITING".equals(e.status)) {
                list.add(new RoomInfo(e.roomCode, e.host.getPlayerName(),
e.status));
            }
        }
        Message m = new Message(Message.Type.ROOM_LIST);
        m.roomList = list;
        return m;
    }

    // ------------------------------------------------------------
    // MATCHMAKING (FIND_MATCH - ghép ngẫu nhiên)
    // ------------------------------------------------------------

    public void enqueueMatchmaking(ClientHandler player) {
        ClientHandler opponent = null;
        synchronized (matchLock) {
            if (waitingForMatch == null) {
                waitingForMatch = player;
            } else if (waitingForMatch == player) {
                return; // đã ở trong hàng đợi rồi, bỏ qua click đúp
            } else {
                opponent = waitingForMatch;
                waitingForMatch = null;
            }
        }

        if (opponent == null) {
            Message waiting = new Message(Message.Type.WAITING);
            waiting.note = "Đang tìm đối thủ...";
            player.sendMessage(waiting);
        } else {
            String code = generateUniqueRoomCode();
            RoomEntry entry = new RoomEntry();
            entry.roomCode = code;
            entry.gameRoom = new GameRoom();
            entry.gameRoom.setRoomCode(code);
            entry.host = opponent;
            entry.guest = player;
            entry.status = "PLAYING";
            rooms.put(code, entry);
            opponent.setCurrentRoomCode(code);
            player.setCurrentRoomCode(code);
            startGameInRoom(entry);
        }
    }

    public void cancelMatchmaking(ClientHandler player) {
        synchronized (matchLock) {
            if (waitingForMatch == player) {
                waitingForMatch = null;
```

```
            }
        }
    }

    /** Gọi khi 1 client ngắt kết nối - dọn dẹp khỏi hàng đợi/phòng chờ nếu có. */
    public synchronized void handleDisconnect(ClientHandler who) {
        synchronized (matchLock) {
            if (waitingForMatch == who) waitingForMatch = null;
        }
        leaveRoom(who);
    }

    // ------------------------------------------------------------
    private void startGameInRoom(RoomEntry entry) {
        entry.host.setPlayerId(1);
        entry.guest.setPlayerId(2);
        entry.gameRoom.setPlayer1(entry.host);
        entry.gameRoom.setPlayer2(entry.guest);
        entry.host.setRoom(entry.gameRoom);
        entry.guest.setRoom(entry.gameRoom);

        Message joined1 = new Message(Message.Type.ROOM_JOINED);
        joined1.roomCode = entry.roomCode;
        joined1.note = entry.guest.getPlayerName();
        entry.host.sendMessage(joined1);

        Message joined2 = new Message(Message.Type.ROOM_JOINED);
        joined2.roomCode = entry.roomCode;
        joined2.note = entry.host.getPlayerName();
        entry.guest.sendMessage(joined2);

        // Gửi kèm tên đối thủ qua Message.Type.NAME cho cả 2 bên NGAY LÚC NÀY.
        // Lý do: cơ chế cũ (ClientHandler.handle case NAME ->
room.notifyNameUpdated)
        // chỉ hoạt động khi "room" đã được gán - nhưng ở luồng Lobby mới, client
        // gửi NAME nhận diện tài khoản NGAY LÚC KẾT NỐI, tức là TRƯỚC khi room
        // được gán (trước cả CREATE_ROOM/FIND_MATCH) - nên tin nhắn NAME ban đầu
        // đó không bao giờ kích hoạt notifyNameUpdated được. Gửi lại ở đây để
        // CaroClient nhận đúng tên đối thủ hiển thị trên tiêu đề/giao diện.
        Message nameForHost = new Message(Message.Type.NAME);
        nameForHost.note = entry.guest.getPlayerName();
        entry.host.sendMessage(nameForHost);

        Message nameForGuest = new Message(Message.Type.NAME);
        nameForGuest.note = entry.host.getPlayerName();
        entry.guest.sendMessage(nameForGuest);

        System.out.println("[RoomManager] Bắt đầu ván tại phòng " + entry.roomCode
                + ": " + entry.host.getPlayerName() + " vs " +
entry.guest.getPlayerName());
        entry.gameRoom.startGame(); // tái sử dụng nguyên vẹn logic GameRoom cũ
    }

    private void fail(ClientHandler client, String reason) {
        Message m = new Message(Message.Type.ROOM_JOIN_FAILED);
        m.note = reason;
        client.sendMessage(m);
    }

    private String generateUniqueRoomCode() {
        String code;
```

```
        do {
            code = generateRoomCode();
        } while (rooms.containsKey(code));
        return code;
    }

    private String generateRoomCode() {
        String chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"; // bỏ ký tự dễ nhầm (I,
O, 0, 1)
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 6; i++) {
            sb.append(chars.charAt(random.nextInt(chars.length())));
        }
        return sb.toString();
    }
}
```

B.5. server/GameRoom.java

```
package caro.server;

import caro.common.Message;
import caro.database.DatabaseConnection;
import caro.database.MatchDAO;
import caro.database.MoveDAO;
import caro.database.UserDAO;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.Timer;
import java.util.TimerTask;

/**
 * GameRoom đại diện cho 1 ván cờ giữa 2 người chơi.
 * Server là "trọng tài": giữ bàn cờ, kiểm tra lượt đi, giới hạn thời gian
 * mỗi lượt, và xác định thắng/thua/hòa.
 * Client KHÔNG tự xử lý thắng thua để tránh lệch trạng thái giữa 2 bên.
 */
public class GameRoom {

    public static final int SIZE = 15; // bàn cờ 15x15

    // Mỗi lượt đánh có tối đa 20 giây; hết giờ server tự đánh thay 1 ô ngẫu
nhiên.
    private static final long TURN_TIME_MS = 20_000;

    private final int[][] board = new int[SIZE][SIZE]; // 0 = trống, 1 = player1,
2 = player2
    private ClientHandler player1;
    private ClientHandler player2;
    private int currentTurn = 1; // player1 (X) luôn đi trước
    private boolean gameOver = false;

    private boolean replayRequested1 = false;
    private boolean replayRequested2 = false;

    // Danh sách toạ độ 5 (hoặc nhiều hơn) ô tạo thành đường thắng, để tô sáng bên
client
    private List<int[]> lastWinCells;

    private final Random random = new Random();
```

```
    private final Timer turnTimer = new Timer(true); // daemon thread, tự tắt khi
server đóng
    private TimerTask currentTurnTask;

    // ==== PHASE 5/7/8: mã phòng + lưu DB + đầu hàng/đề nghị hòa ====
    private String roomCode;
    private Integer matchId;       // null nếu 1 trong 2 người chơi chưa đăng nhập
(anonymous demo)
    private int moveCounter = 0;
    private long startTimeMillis;
    private Integer drawOfferFromPlayerId; // null nếu không có đề nghị hòa nào
đang chờ

    public void setRoomCode(String roomCode) { this.roomCode = roomCode; }

    public void setPlayer1(ClientHandler p) { this.player1 = p; }
    public void setPlayer2(ClientHandler p) { this.player2 = p; }

    public synchronized void startGame() {
        cancelTurnTimer();
        gameOver = false;
        currentTurn = 1;
        replayRequested1 = false;
        replayRequested2 = false;
        drawOfferFromPlayerId = null;
        moveCounter = 0;
        startTimeMillis = System.currentTimeMillis();
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                board[i][j] = 0;
            }
        }

        // PHASE 8: tạo bản ghi match trong DB nếu cả 2 người chơi đều đã đăng
nhập.
        // Nếu 1 trong 2 là kết nối "vô danh" (chạy CaroClient.java demo cũ, chưa
        // qua LoginFrame), matchId = null -> vẫn chơi bình thường, chỉ không lưu
DB.
        matchId = null;
        Integer uid1 = player1.getUserId();
        Integer uid2 = player2.getUserId();
        if (uid1 != null && uid2 != null) {
            try {
                matchId = new MatchDAO().createMatch(uid1, uid2, "ONLINE",
roomCode);
            } catch (DatabaseConnection.DatabaseException e) {
                System.out.println("[GameRoom] Không thể tạo match trong DB: " +
e.getMessage());
            }
        }

        Message m1 = new Message(Message.Type.START);
        m1.playerId = 1;
        player1.sendMessage(m1);

        Message m2 = new Message(Message.Type.START);
        m2.playerId = 2;
        player2.sendMessage(m2);

        scheduleTurnTimer();
    }
```

```
    /**
     * Gọi khi 1 ClientHandler nhận được tên người chơi (Message.Type.NAME),
     * chuyển tiếp tên đó cho đối thủ để hiển thị lên GUI.
     */
    public synchronized void notifyNameUpdated(ClientHandler who) {
        ClientHandler other = (who == player1) ? player2 : player1;
        if (other == null) return;
        Message m = new Message(Message.Type.NAME);
        m.note = who.getPlayerName();
        other.sendMessage(m);
    }

    /**
     * Xử lý 1 nước đi do ClientHandler chuyển lên (người chơi tự bấm).
     */
    public synchronized void handleMove(int playerId, int x, int y) {
        if (gameOver) return;
        if (playerId != currentTurn) return;              // không đúng lượt -> bỏ
qua
        if (x < 0 || y < 0 || x >= SIZE || y >= SIZE) return;
        if (board[x][y] != 0) return;                      // ô đã có quân
        applyMove(playerId, x, y);
    }

    /**
     * Gọi khi hết giờ 1 lượt mà người chơi chưa đánh - server tự chọn
     * 1 ô trống ngẫu nhiên để đánh thay, giữ ván cờ không bị treo mãi.
     */
    private synchronized void autoMove() {
        if (gameOver) return;

        List<int[]> emptyCells = new ArrayList<int[]>();
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                if (board[i][j] == 0) emptyCells.add(new int[]{i, j});
            }
        }
        if (emptyCells.isEmpty()) return; // hết ô trống thì lẽ ra đã hòa từ trước

        int[] chosen = emptyCells.get(random.nextInt(emptyCells.size()));
        System.out.println("[Server] Hết giờ! Tự động đánh thay Player " +
currentTurn
                + " tại (" + chosen[0] + "," + chosen[1] + ")");
        applyMove(currentTurn, chosen[0], chosen[1]);
    }

    /**
     * Logic đặt quân dùng chung cho cả nước đi thật (handleMove) và
     * nước đi tự động khi hết giờ (autoMove).
     */
    private void applyMove(int playerId, int x, int y) {
        board[x][y] = playerId;
        currentTurn = (playerId == 1) ? 2 : 1;
        moveCounter++;

        if (matchId != null) {
            Integer uid = (playerId == 1) ? player1.getUserId() :
player2.getUserId();
            if (uid != null) {
                try {
```

```
                    new MoveDAO().saveMove(matchId, uid, x, y, moveCounter);
                } catch (DatabaseConnection.DatabaseException e) {
                    // Lỗi lưu 1 nước đi KHÔNG được làm gián đoạn ván đang chơi
(mục XXIV)
                    System.out.println("[GameRoom] Lỗi lưu nước đi vào DB: " +
e.getMessage());
                }
            }
        }

        Message move = new Message(Message.Type.MOVE);
        move.playerId = playerId;
        move.x = x;
        move.y = y;
        move.nextTurn = currentTurn;
        broadcast(move);

        if (checkWin(x, y, playerId)) {
            gameOver = true;
            cancelTurnTimer();
            finishMatchInDb(playerId, "WIN");
            Message win = new Message(Message.Type.WIN);
            win.winnerId = playerId;
            if (lastWinCells != null) {
                int n = lastWinCells.size();
                win.winLineX = new int[n];
                win.winLineY = new int[n];
                for (int i = 0; i < n; i++) {
                    win.winLineX[i] = lastWinCells.get(i)[0];
                    win.winLineY[i] = lastWinCells.get(i)[1];
                }
            }
            broadcast(win);
            return;
        }

        if (isBoardFull()) {
            gameOver = true;
            cancelTurnTimer();
            finishMatchInDb(null, "DRAW");
            broadcast(new Message(Message.Type.DRAW));
            return;
        }

        // Ván vẫn tiếp tục -> bắt đầu đếm giờ cho lượt kế tiếp
        scheduleTurnTimer();
    }

    /**
     * Ghi kết quả cuối cùng của trận vào DB (bảng matches) và cập nhật
     * thống kê/rating của cả 2 người chơi (bảng users) - mục XIX.
     * Bỏ qua hoàn toàn nếu matchId == null (1 trong 2 bên không đăng nhập).
     *
     * @param winnerPlayerId 1 hoặc 2 nếu có người thắng rõ ràng, null nếu hòa
     * @param result         "WIN" / "DRAW" - ghi vào cột matches.result
     */
    private void finishMatchInDb(Integer winnerPlayerId, String result) {
        if (matchId == null) return;
        try {
            Integer uid1 = player1.getUserId();
            Integer uid2 = player2.getUserId();
```

```
            Integer winnerUserId = null;
            if (winnerPlayerId != null) {
                winnerUserId = (winnerPlayerId == 1) ? uid1 : uid2;
            }
            int durationSec = (int) Math.max(0, (System.currentTimeMillis() -
startTimeMillis) / 1000);

            new MatchDAO().finishMatch(matchId, winnerUserId, result,
durationSec);

            UserDAO userDAO = new UserDAO();
            String outcome1, outcome2;
            if ("DRAW".equals(result)) {
                outcome1 = "DRAW";
                outcome2 = "DRAW";
            } else {
                outcome1 = (winnerPlayerId != null && winnerPlayerId == 1) ? "WIN"
: "LOSS";
                outcome2 = (winnerPlayerId != null && winnerPlayerId == 2) ? "WIN"
: "LOSS";
            }
            userDAO.updateStatsAfterMatch(uid1, outcome1, player2.getRating());
            userDAO.updateStatsAfterMatch(uid2, outcome2, player1.getRating());
        } catch (DatabaseConnection.DatabaseException e) {
            System.out.println("[GameRoom] Lỗi lưu kết quả trận đấu vào DB: " +
e.getMessage());
        }
    }

    private void scheduleTurnTimer() {
        cancelTurnTimer();
        currentTurnTask = new TimerTask() {
            @Override
            public void run() {
                autoMove();
            }
        };
        turnTimer.schedule(currentTurnTask, TURN_TIME_MS);
    }

    private void cancelTurnTimer() {
        if (currentTurnTask != null) {
            currentTurnTask.cancel();
            currentTurnTask = null;
        }
    }

    private boolean isBoardFull() {
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                if (board[i][j] == 0) return false;
            }
        }
        return true;
    }

    /**
     * Kiểm tra 5 quân liên tiếp tính từ ô (x,y) vừa đánh,
     * theo 4 hướng: ngang, dọc, chéo xuống, chéo lên.
     * Nếu thắng, lưu lại toạ độ các ô thắng vào lastWinCells.
     */
```

```
    private boolean checkWin(int x, int y, int playerId) {
        int[][] directions = { {1, 0}, {0, 1}, {1, 1}, {1, -1} };
        for (int[] d : directions) {
            List<int[]> line = new ArrayList<int[]>();
            line.add(new int[]{x, y});
            collectDirection(x, y, d[0], d[1], playerId, line);
            collectDirection(x, y, -d[0], -d[1], playerId, line);
            if (line.size() >= 5) {
                lastWinCells = line;
                return true;
            }
        }
        return false;
    }

    private void collectDirection(int x, int y, int dx, int dy, int playerId,
List<int[]> line) {
        int nx = x + dx, ny = y + dy;
        while (nx >= 0 && ny >= 0 && nx < SIZE && ny < SIZE && board[nx][ny] ==
playerId) {
            line.add(new int[]{nx, ny});
            nx += dx;
            ny += dy;
        }
    }

    private void broadcast(Message m) {
        player1.sendMessage(m);
        player2.sendMessage(m);
    }

    /**
     * Gọi khi 1 người chơi bấm nút "Chơi lại".
     * Chỉ khi CẢ HAI người cùng bấm thì ván mới mới bắt đầu.
     */
    public synchronized void requestReplay(int playerId) {
        if (!gameOver) return; // chỉ cho xin chơi lại khi ván trước đã kết thúc

        if (playerId == 1) {
            replayRequested1 = true;
        } else {
            replayRequested2 = true;
        }

        if (replayRequested1 && replayRequested2) {
            startGame();
        } else {
            // Báo cho người còn lại biết đối thủ đã muốn chơi lại
            ClientHandler waitingFor = (playerId == 1) ? player2 : player1;
            Message note = new Message(Message.Type.REPLAY);
            note.note = "Đối thủ muốn chơi lại! Bấm \"Chơi lại\" để bắt đầu ván
mới.";
            if (waitingFor != null) {
                waitingFor.sendMessage(note);
            }
        }
    }

    /**
     * Chuyển tiếp tin nhắn chat từ 1 người chơi tới cả 2 người
     * (kể cả người gửi, để đơn giản hóa - client nhận lại đúng tin của mình
```

```
     * kèm tên, không cần tự hiển thị cục bộ trước).
     */
    public synchronized void broadcastChat(ClientHandler sender, String text) {
        if (text == null || text.trim().isEmpty()) return;
        Message chat = new Message(Message.Type.CHAT);
        chat.note = sender.getPlayerName() + ": " + text.trim();
        if (player1 != null) player1.sendMessage(chat);
        if (player2 != null) player2.sendMessage(chat);
    }

    /**
     * Gọi khi 1 trong 2 client ngắt kết nối giữa ván.
     * Người còn lại được tính thắng (ABANDONED) - phù hợp kỳ vọng thông thường
     * "đối thủ thoát thì mình thắng", đồng thời vẫn phân biệt được với thắng
     * bình thường qua cột matches.result = 'ABANDONED'.
     */
    public synchronized void notifyOpponentLeft(ClientHandler leaver) {
        if (gameOver) return;
        gameOver = true;
        cancelTurnTimer();
        ClientHandler other = (leaver == player1) ? player2 : player1;
        Integer winnerPlayerId = (other == player1) ? 1 : (other == player2) ? 2 :
null;
        finishMatchInDb(winnerPlayerId, "ABANDONED");
        if (other != null) {
            other.sendMessage(new Message(Message.Type.OPPONENT_LEFT));
        }
    }

    /**
     * PHASE 7: người chơi bấm "Đầu hàng" - đối thủ thắng ngay lập tức.
     */
    public synchronized void surrender(int playerId) {
        if (gameOver) return;
        gameOver = true;
        cancelTurnTimer();
        int winnerId = (playerId == 1) ? 2 : 1;
        finishMatchInDb(winnerId, "WIN");

        Message win = new Message(Message.Type.WIN);
        win.winnerId = winnerId;
        win.note = "Đối thủ đã đầu hàng.";
        broadcast(win);
    }

    /**
     * PHASE 7: người chơi bấm "Đề nghị hòa" - chuyển tiếp đề nghị cho đối thủ.
     */
    public synchronized void offerDraw(int playerId) {
        if (gameOver) return;
        drawOfferFromPlayerId = playerId;
        ClientHandler offerer = (playerId == 1) ? player1 : player2;
        ClientHandler other = (playerId == 1) ? player2 : player1;
        if (other == null) return;
        Message m = new Message(Message.Type.OFFER_DRAW);
        m.note = offerer.getPlayerName() + " đề nghị hòa.";
        other.sendMessage(m);
    }

    /**
     * PHASE 7: đối thủ phản hồi đề nghị hòa.
```

```
     * @param playerId người PHẢN HỒI (không phải người đề nghị)
     * @param accept   true = đồng ý hòa, false = từ chối
     */
    public synchronized void respondDraw(int playerId, boolean accept) {
        if (gameOver || drawOfferFromPlayerId == null) return;
        if (playerId == drawOfferFromPlayerId) return; // người đề nghị không tự
phản hồi được

        if (accept) {
            gameOver = true;
            cancelTurnTimer();
            finishMatchInDb(null, "DRAW");
            broadcast(new Message(Message.Type.DRAW));
        } else {
            ClientHandler offerer = (drawOfferFromPlayerId == 1) ? player1 :
player2;
            if (offerer != null) {
                Message m = new Message(Message.Type.DRAW_REJECT);
                m.note = "Đối thủ đã từ chối đề nghị hòa.";
                offerer.sendMessage(m);
            }
        }
        drawOfferFromPlayerId = null;
    }
}
```

B.6. server/AuthServer.java

```
package caro.server;

import caro.common.Message;

import java.io.EOFException;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketException;

/**
 * Server RIÊNG cho việc xác thực (đăng nhập / đăng ký), chạy trên cổng
 * {@link #AUTH_PORT} - HOÀN TOÀN TÁCH BIỆT với cổng chơi game 12345 mà
 * CaroServer/ClientHandler/GameRoom đang dùng.
 *
 * LÝ DO TÁCH RIÊNG (thay vì nhét LOGIN/REGISTER vào ClientHandler cũ):
 *   CaroServer.main() hiện tại accept() ĐÚNG 2 lần rồi lập tức ghép thành
 *   1 GameRoom (xem code gốc). Nếu để LOGIN/REGISTER đi chung cổng đó, kết
 *   nối "chỉ để đăng nhập" sẽ bị server hiểu nhầm là 1 trong 2 người chơi,
 *   làm treo hoặc vỡ luồng ghép phòng đang chạy rất ổn định.
 *   => Tách cổng là cách AN TOÀN NHẤT để thêm tính năng mới mà không đụng
 *      1 dòng nào trong CaroServer/ClientHandler/GameRoom hiện tại.
 *
 * Ở Phase 5 (RoomManager đa phòng), khi CaroServer được viết lại để spawn
 * 1 thread ngay khi accept() (thay vì chờ đủ 2 người), AuthServer có thể
 * được gộp lại chung 1 cổng với logic phòng/lobby nếu muốn - nhưng đó là
 * quyết định của Phase 5, không phải bây giờ.
 */
public class AuthServer {

    public static final int AUTH_PORT = 12346;

    /**
```

```
     * Khởi động AuthServer trên 1 Thread nền (daemon), để không chặn
     * CaroServer.main() tiếp tục chạy vòng lặp game như bình thường.
     * Gọi 1 lần duy nhất từ CaroServer.main().
     */
    public static void startInBackground() {
        Thread t = new Thread(AuthServer::runAcceptLoop, "AuthServer");
        t.setDaemon(true);
        t.start();
    }

    private static void runAcceptLoop() {
        try (ServerSocket serverSocket = new ServerSocket(AUTH_PORT)) {
            System.out.println("[AuthServer] Đang lắng nghe tại port " + AUTH_PORT
                    + " (xử lý đăng nhập/đăng ký)...");
            while (true) {
                Socket socket = serverSocket.accept();
                Thread handlerThread = new Thread(new AuthClientHandler(socket));
                handlerThread.setDaemon(true);
                handlerThread.start();
            }
        } catch (IOException e) {
            System.out.println("[AuthServer] Không thể khởi động ở port " +
AUTH_PORT
                    + ": " + e.getMessage());
            System.out.println("[AuthServer] Chức năng đăng nhập/đăng ký sẽ KHÔNG
hoạt động, "
                    + "nhưng chơi game (port 12345) vẫn chạy bình thường.");
        }
    }

    /**
     * Xử lý 1 kết nối xác thực. Cho phép nhiều lần thử trên cùng 1 kết nối
     * (ví dụ: đăng ký thất bại vì trùng username -> thử lại ngay, không
     * cần LoginFrame/RegisterFrame phải mở lại socket mới).
     */
    private static class AuthClientHandler implements Runnable {
        private final Socket socket;
        private final AuthService authService = new AuthService();

        AuthClientHandler(Socket socket) {
            this.socket = socket;
        }

        @Override
        public void run() {
            try (Socket s = socket) {
                ObjectOutputStream out = new
ObjectOutputStream(s.getOutputStream());
                out.flush();
                ObjectInputStream in = new ObjectInputStream(s.getInputStream());

                Message request;
                while ((request = (Message) in.readObject()) != null) {
                    Message response = handle(request);
                    if (response != null) {
                        out.writeObject(response);
                        out.flush();
                        out.reset();
                    }
                }
            } catch (EOFException | SocketException e) {
```

```
                // Client đóng kết nối sau khi xác thực xong - bình thường, không
cần log lỗi
            } catch (Exception e) {
                System.out.println("[AuthServer] Lỗi xử lý kết nối: " +
e.getMessage());
            }
        }

        private Message handle(Message request) {
            switch (request.type) {
                case LOGIN:
                    System.out.println("[AuthServer] Yêu cầu đăng nhập: " +
request.username);
                    return authService.login(request.username, request.password);

                case REGISTER:
                    System.out.println("[AuthServer] Yêu cầu đăng ký: " +
request.username);
                    return authService.register(request.username,
request.password, request.displayName);

                default:
                    Message err = new Message(Message.Type.ERROR);
                    err.note = "AuthServer chỉ hỗ trợ LOGIN/REGISTER.";
                    return err;
            }
        }
    }

    private AuthServer() {
    }
}
```

B.7. server/AuthService.java

```
package caro.server;

import caro.common.Message;
import caro.common.User;
import caro.database.DatabaseConnection;
import caro.database.UserDAO;

/**
 * Business logic cho đăng nhập / đăng ký.
 * Đây là nơi DUY NHẤT chuyển đổi giữa "Message" (giao thức mạng) và
 * "UserDAO" (tầng dữ liệu) - AuthServer/ClientHandler không gọi UserDAO
 * trực tiếp, luôn đi qua AuthService để business rule tập trung 1 chỗ.
 *
 * Toàn bộ validate ở đây là kiểm tra PHÍA SERVER (không tin dữ liệu client
 * gửi lên, dù RegisterFrame/LoginFrame đã validate ở client rồi).
 */
public class AuthService {

    private final UserDAO userDAO = new UserDAO();

    /**
     * Xử lý 1 request LOGIN, trả về Message sẵn sàng gửi ngược lại client:
     * LOGIN_SUCCESS (kèm user) hoặc LOGIN_FAILED (kèm note = lý do).
     */
    public Message login(String username, String password) {
        if (isBlank(username) || isBlank(password)) {
            return failed(Message.Type.LOGIN_FAILED, "Vui lòng nhập đầy đủ tên
đăng nhập và mật khẩu.");
```

```
        }
        try {
            User user = userDAO.authenticate(username.trim(), password);
            if (user == null) {
                return failed(Message.Type.LOGIN_FAILED, "Sai tên đăng nhập hoặc
mật khẩu.");
            }
            userDAO.updateStatus(user.getId(), "ONLINE");
            Message ok = new Message(Message.Type.LOGIN_SUCCESS);
            ok.user = user;
            return ok;
        } catch (DatabaseConnection.DatabaseException e) {
            return failed(Message.Type.LOGIN_FAILED, e.getMessage());
        }
    }

    /**
     * Xử lý 1 request REGISTER, trả về Message sẵn sàng gửi ngược lại client:
     * REGISTER_SUCCESS (kèm user) hoặc REGISTER_FAILED (kèm note = lý do).
     */
    public Message register(String username, String password, String displayName)
{
        String validationError = validateRegister(username, password,
displayName);
        if (validationError != null) {
            return failed(Message.Type.REGISTER_FAILED, validationError);
        }
        try {
            String finalDisplayName = isBlank(displayName) ? username.trim() :
displayName.trim();
            User user = userDAO.register(username.trim(), password,
finalDisplayName);
            Message ok = new Message(Message.Type.REGISTER_SUCCESS);
            ok.user = user;
            return ok;
        } catch (DatabaseConnection.DatabaseException e) {
            return failed(Message.Type.REGISTER_FAILED, e.getMessage());
        }
    }

    /**
     * Validate phía server - PHẢI kiểm tra lại dù client đã kiểm tra rồi,
     * vì client có thể bị sửa đổi / bị bypass (theo nguyên tắc "không tin
     * dữ liệu từ Client" - mục XXV của yêu cầu).
     *
     * @return null nếu hợp lệ, hoặc chuỗi mô tả lỗi đầu tiên gặp phải
     */
    private String validateRegister(String username, String password, String
displayName) {
        if (isBlank(username) || isBlank(password)) {
            return "Vui lòng nhập đầy đủ thông tin bắt buộc.";
        }
        String trimmedUsername = username.trim();
        if (trimmedUsername.length() < 3) {
            return "Tên đăng nhập phải có ít nhất 3 ký tự.";
        }
        if (trimmedUsername.length() > 32) {
            return "Tên đăng nhập không được vượt quá 32 ký tự.";
        }
        if (!trimmedUsername.matches("[a-zA-Z0-9_]+")) {
            return "Tên đăng nhập chỉ được chứa chữ cái, số và dấu gạch dưới.";
```

```
        }
        if (password.length() < 6) {
            return "Mật khẩu phải có ít nhất 6 ký tự.";
        }
        if (displayName != null && displayName.trim().length() > 64) {
            return "Tên hiển thị không được vượt quá 64 ký tự.";
        }
        return null;
    }

    private boolean isBlank(String s) {
        return s == null || s.trim().isEmpty();
    }

    private Message failed(Message.Type type, String reason) {
        Message m = new Message(type);
        m.note = reason;
        return m;
    }
}
```

B.8. database/UserDAO.java

```
package caro.database;

import caro.common.User;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;

/**
 * DAO (Data Access Object) cho bảng "users".
 * Toàn bộ thao tác đăng ký / đăng nhập / cập nhật thống kê sau trận /
 * bảng xếp hạng đều đi qua class này - không có nơi nào khác trong
 * project được phép chạy SQL trực tiếp lên bảng users.
 */
public class UserDAO {

    /** Rating mặc định khi tạo tài khoản mới. */
    private static final int DEFAULT_RATING = 1000;

    // ------------------------------------------------------------
    // Băm mật khẩu (SHA-256) - không dùng thư viện ngoài để tương
    // thích tuyệt đối với Java 8 + NetBeans 8.0.2 không cần thêm JAR.
    // ------------------------------------------------------------
    private String hashPassword(String plainPassword) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hashBytes = digest.digest(plainPassword.getBytes("UTF-8"));
            StringBuilder sb = new StringBuilder();
            for (byte b : hashBytes) {
                String hex = Integer.toHexString(0xff & b);
                if (hex.length() == 1) sb.append('0');
                sb.append(hex);
            }
```

```
            return sb.toString();
        } catch (NoSuchAlgorithmException | java.io.UnsupportedEncodingException
e) {
            // SHA-256 và UTF-8 luôn có sẵn trên mọi JVM chuẩn -> thực tế không
xảy ra
            throw new RuntimeException("Lỗi hệ thống khi băm mật khẩu", e);
        }
    }

    // ------------------------------------------------------------
    // ĐĂNG KÝ
    // ------------------------------------------------------------

    /** @return true nếu username đã tồn tại (không phân biệt hoa thường vì cột
dùng collation *_ci) */
    public boolean isUsernameTaken(String username) throws
DatabaseConnection.DatabaseException {
        String sql = "SELECT id FROM users WHERE username = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, username);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next();
            }
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi kiểm tra username: " + e.getMessage(), e);
        }
    }

    /**
     * Tạo tài khoản mới. KHÔNG kiểm tra trùng username ở đây (caller nên
     * gọi isUsernameTaken trước để hiển thị lỗi sớm); tuy nhiên cột
     * username có UNIQUE KEY ở database nên nếu có race-condition giữa
     * 2 request đăng ký cùng username, MySQL sẽ tự chặn và SQLException
     * (mã lỗi trùng khoá) sẽ được ném ra dưới dạng DatabaseException.
     *
     * @return đối tượng User vừa tạo (đã có id do MySQL tự sinh)
     */
    public User register(String username, String plainPassword, String
displayName)
            throws DatabaseConnection.DatabaseException {
        String sql = "INSERT INTO users (username, password, display_name, rating)
"
                + "VALUES (?, ?, ?, ?)";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, username);
            ps.setString(2, hashPassword(plainPassword));
            ps.setString(3, displayName);
            ps.setInt(4, DEFAULT_RATING);
            ps.executeUpdate();

            try (ResultSet keys = ps.getGeneratedKeys()) {
                int newId = keys.next() ? keys.getInt(1) : -1;
                return new User(newId, username, displayName, 0, 0, 0, 0,
DEFAULT_RATING);
            }
        } catch (SQLException e) {
            if ("23000".equals(e.getSQLState())) {
```

```
                throw new DatabaseConnection.DatabaseException(
                        "Tên đăng nhập \"" + username + "\" đã tồn tại. Vui lòng
chọn tên khác.", e);
            }
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi đăng ký tài khoản: " + e.getMessage(), e);
        }
    }

    // ------------------------------------------------------------
    // ĐĂNG NHẬP
    // ------------------------------------------------------------

    /**
     * Kiểm tra username/password. Trả về User nếu đúng, null nếu sai
     * username hoặc sai password (không phân biệt 2 trường hợp này khi
     * trả kết quả, để tránh lộ thông tin username nào tồn tại).
     */
    public User authenticate(String username, String plainPassword)
            throws DatabaseConnection.DatabaseException {
        String sql = "SELECT id, username, password, display_name, wins, losses,
draws, "
                + "total_games, rating, created_at FROM users WHERE username = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, username);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return null; // sai username

                String storedHash = rs.getString("password");
                String inputHash = hashPassword(plainPassword);
                if (!storedHash.equals(inputHash)) return null; // sai password

                User user = mapRow(rs);
                touchLastLogin(user.getId());
                return user;
            }
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi đăng nhập: " + e.getMessage(), e);
        }
    }

    private void touchLastLogin(int userId) throws
DatabaseConnection.DatabaseException {
        String sql = "UPDATE users SET last_login = ? WHERE id = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setTimestamp(1, new Timestamp(System.currentTimeMillis()));
            ps.setInt(2, userId);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi cập nhật lần đăng nhập cuối: " + e.getMessage(), e);
        }
    }

    // ------------------------------------------------------------
    // TRUY VẤN
    // ------------------------------------------------------------
```

```
    public User getById(int userId) throws DatabaseConnection.DatabaseException {
        String sql = "SELECT id, username, password, display_name, wins, losses,
draws, "
                + "total_games, rating, created_at FROM users WHERE id = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? mapRow(rs) : null;
            }
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi tải thông tin người dùng: " + e.getMessage(), e);
        }
    }

    /** Bảng xếp hạng: sắp xếp theo rating giảm dần, giới hạn số dòng trả về. */
    public List<User> getLeaderboard(int limit) throws
DatabaseConnection.DatabaseException {
        String sql = "SELECT id, username, password, display_name, wins, losses,
draws, "
                + "total_games, rating, created_at FROM users "
                + "ORDER BY rating DESC, wins DESC LIMIT ?";
        List<User> result = new ArrayList<User>();
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, limit);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    result.add(mapRow(rs));
                }
            }
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi tải bảng xếp hạng: " + e.getMessage(), e);
        }
        return result;
    }

    // ------------------------------------------------------------
    // CẬP NHẬT SAU TRẬN ĐẤU
    // ------------------------------------------------------------

    /** Hệ số K dùng cho công thức Elo đơn giản hoá. */
    private static final int RATING_K = 20;

    /**
     * Cập nhật thống kê (wins/losses/draws/total_games) và rating đơn giản
     * kiểu Elo cho 1 người chơi sau khi trận đấu kết thúc.
     *
     * @param userId        id người chơi cần cập nhật
     * @param outcome       "WIN" / "LOSS" / "DRAW"
     * @param opponentRating rating hiện tại của đối thủ (dùng để tính Elo);
     *                       truyền DEFAULT_RATING nếu chơi với AI hoặc không có
đối thủ thật
     */
    public void updateStatsAfterMatch(int userId, String outcome, int
opponentRating)
            throws DatabaseConnection.DatabaseException {
        String sql = "SELECT rating FROM users WHERE id = ?";
        int currentRating = DEFAULT_RATING;
```

```
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) currentRating = rs.getInt("rating");
            }
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi đọc rating hiện tại: " + e.getMessage(), e);
        }

        double actualScore; // 1 = thắng, 0.5 = hòa, 0 = thua
        String updateColumn;
        if ("WIN".equals(outcome)) {
            actualScore = 1.0;
            updateColumn = "wins";
        } else if ("DRAW".equals(outcome)) {
            actualScore = 0.5;
            updateColumn = "draws";
        } else {
            actualScore = 0.0;
            updateColumn = "losses";
        }

        double expectedScore = 1.0 / (1.0 + Math.pow(10, (opponentRating -
currentRating) / 400.0));
        int ratingDelta = (int) Math.round(RATING_K * (actualScore -
expectedScore));
        int newRating = Math.max(0, currentRating + ratingDelta);

        String update = "UPDATE users SET " + updateColumn + " = " + updateColumn
+ " + 1, "
                + "total_games = total_games + 1, rating = ? WHERE id = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(update)) {
            ps.setInt(1, newRating);
            ps.setInt(2, userId);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi cập nhật thống kê sau trận: " + e.getMessage(), e);
        }
    }

    /** Cập nhật trạng thái online (ONLINE/IN_LOBBY/IN_ROOM/PLAYING/OFFLINE). */
    public void updateStatus(int userId, String status) throws
DatabaseConnection.DatabaseException {
        String sql = "UPDATE users SET status = ? WHERE id = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, status);
            ps.setInt(2, userId);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi cập nhật trạng thái: " + e.getMessage(), e);
        }
    }

    /** Đổi display name (dùng cho màn hình Hồ sơ - Phase sau). */
    public void updateDisplayName(int userId, String newDisplayName)
```

```
            throws DatabaseConnection.DatabaseException {
        String sql = "UPDATE users SET display_name = ? WHERE id = ?";
        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, newDisplayName);
            ps.setInt(2, userId);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new DatabaseConnection.DatabaseException(
                    "Lỗi khi cập nhật tên hiển thị: " + e.getMessage(), e);
        }
    }

    // ------------------------------------------------------------
    private User mapRow(ResultSet rs) throws SQLException {
        User u = new User();
        u.setId(rs.getInt("id"));
        u.setUsername(rs.getString("username"));
        u.setDisplayName(rs.getString("display_name"));
        u.setWins(rs.getInt("wins"));
        u.setLosses(rs.getInt("losses"));
        u.setDraws(rs.getInt("draws"));
        u.setTotalGames(rs.getInt("total_games"));
        u.setRating(rs.getInt("rating"));
        u.setCreatedAt(rs.getTimestamp("created_at"));
        return u;
    }
}
```

B.9. ai/MediumAI.java

```
package caro.ai;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

/**
 * AI mức TRUNG BÌNH:
 *   1. Nếu có nước đi giúp AI thắng ngay -> đánh luôn.
 *   2. Nếu đối thủ sắp thắng (còn 1 nước là đủ 5) -> chặn ngay ô đó.
 *   3. Ngược lại, ưu tiên ô tạo được chuỗi liên tiếp DÀI NHẤT cho AI
 *      (khuyến khích tạo chuỗi 3, 4 quân).
 *   4. Nếu bàn cờ trống hoàn toàn -> đánh vào giữa bàn.
 */
public class MediumAI implements CaroAI {

    private final Random random = new Random();

    @Override
    public int[] nextMove(int[][] board, int aiPlayerId, int humanPlayerId) {
        List<int[]> emptyCells = collectEmptyCells(board);
        if (emptyCells.isEmpty()) return null;

        if (isBoardEmpty(board)) {
            return new int[]{SIZE / 2, SIZE / 2};
        }

        for (int[] cell : emptyCells) {
            if (wouldWin(board, cell[0], cell[1], aiPlayerId)) return cell;
        }

        for (int[] cell : emptyCells) {
```

```
            if (wouldWin(board, cell[0], cell[1], humanPlayerId)) return cell;
        }

        int[] best = null;
        int bestScore = -1;
        for (int[] cell : emptyCells) {
            int score = maxLineIfPlaced(board, cell[0], cell[1], aiPlayerId);
            if (score > bestScore) {
                bestScore = score;
                best = cell;
            }
        }
        return best != null ? best :
emptyCells.get(random.nextInt(emptyCells.size()));
    }

    private boolean isBoardEmpty(int[][] board) {
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                if (board[i][j] != 0) return false;
            }
        }
        return true;
    }

    private List<int[]> collectEmptyCells(int[][] board) {
        List<int[]> list = new ArrayList<int[]>();
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                if (board[i][j] == 0) list.add(new int[]{i, j});
            }
        }
        return list;
    }

    /** Nếu đặt "player" vào (x,y) thì có tạo đủ 5 quân liên tiếp không? */
    static boolean wouldWin(int[][] board, int x, int y, int player) {
        return maxLineIfPlaced(board, x, y, player) >= 5;
    }

    /** Nếu đặt "player" vào (x,y), chuỗi liên tiếp DÀI NHẤT (theo 1 trong 4
hướng) là bao nhiêu? */
    static int maxLineIfPlaced(int[][] board, int x, int y, int player) {
        int[][] directions = {{1, 0}, {0, 1}, {1, 1}, {1, -1}};
        int best = 1;
        for (int[] d : directions) {
            int count = 1;
            count += countDirection(board, x, y, d[0], d[1], player);
            count += countDirection(board, x, y, -d[0], -d[1], player);
            if (count > best) best = count;
        }
        return best;
    }

    private static int countDirection(int[][] board, int x, int y, int dx, int dy,
int player) {
        int count = 0;
        int nx = x + dx, ny = y + dy;
        while (nx >= 0 && ny >= 0 && nx < SIZE && ny < SIZE && board[nx][ny] ==
player) {
            count++;
```

```
            nx += dx;
            ny += dy;
        }
        return count;
    }
}
```

B.10. ai/HardAI.java

```
package caro.ai;

import java.util.ArrayList;
import java.util.List;

/**
 * AI mức KHÓ: dùng heuristic chấm điểm theo pattern (giống hướng dẫn "dùng
 * heuristic hoặc Minimax + Alpha-Beta" - ở đây chọn heuristic scoring vì
 * bàn 15x15 quá lớn để Minimax đầy đủ chạy đủ nhanh mà vẫn dễ hiểu cho
 * sinh viên trình bày).
 *
 * Với MỖI ô trống (chỉ xét các ô gần quân cờ đã có, để tăng tốc), tính:
 *   - điểm TẤN CÔNG: nếu AI đặt vào đây thì các chuỗi AI tạo được mạnh cỡ nào
 *   - điểm PHÒNG THỦ: nếu ĐỐI THỦ đặt vào đây thì họ tạo được chuỗi mạnh cỡ nào
 *     (điểm phòng thủ cao nghĩa là ô đó QUAN TRỌNG PHẢI CHẶN)
 * Tổng 2 điểm lại, chọn ô có tổng điểm cao nhất.
 */
public class HardAI implements CaroAI {

    private static final int SEARCH_RADIUS = 2;

    @Override
    public int[] nextMove(int[][] board, int aiPlayerId, int humanPlayerId) {
        List<int[]> candidates = collectCandidateCells(board);
        if (candidates.isEmpty()) return null;

        int[] best = null;
        long bestScore = Long.MIN_VALUE;
        for (int[] cell : candidates) {
            long offense = evaluateCell(board, cell[0], cell[1], aiPlayerId);
            long defense = evaluateCell(board, cell[0], cell[1], humanPlayerId);
            long total = offense + (long) (defense * 1.05); // ưu tiên phòng thủ
nhỉnh hơn 1 chút
            if (total > bestScore) {
                bestScore = total;
                best = cell;
            }
        }
        return best;
    }

    /**
     * Chỉ xét các ô trống nằm trong bán kính SEARCH_RADIUS quanh 1 quân cờ
     * bất kỳ đã có trên bàn - giúp AI chạy nhanh trên bàn 15x15 (225 ô)
     * thay vì phải chấm điểm toàn bộ ô trống mỗi lượt.
     * Nếu bàn cờ trống hoàn toàn (nước đi đầu tiên) -> trả về ô giữa bàn.
     */
    private List<int[]> collectCandidateCells(int[][] board) {
        boolean[][] isCandidate = new boolean[SIZE][SIZE];
        boolean anyStone = false;
        List<int[]> result = new ArrayList<int[]>();

        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
```

```
                if (board[i][j] != 0) {
                    anyStone = true;
                    for (int di = -SEARCH_RADIUS; di <= SEARCH_RADIUS; di++) {
                        for (int dj = -SEARCH_RADIUS; dj <= SEARCH_RADIUS; dj++) {
                            int ni = i + di, nj = j + dj;
                            if (ni >= 0 && nj >= 0 && ni < SIZE && nj < SIZE
                                    && board[ni][nj] == 0 && !isCandidate[ni][nj])
{
                                isCandidate[ni][nj] = true;
                                result.add(new int[]{ni, nj});
                            }
                        }
                    }
                }
            }
        }

        if (!anyStone) {
            result.add(new int[]{SIZE / 2, SIZE / 2});
        }
        return result;
    }

    /** Tổng điểm heuristic của ô (x,y) nếu "player" đặt quân vào đó, cộng dồn cả
4 hướng. */
    private long evaluateCell(int[][] board, int x, int y, int player) {
        int[][] axes = {{1, 0}, {0, 1}, {1, 1}, {1, -1}};
        long total = 0;
        for (int[] axis : axes) {
            total += lineScore(board, x, y, axis[0], axis[1], player);
        }
        return total;
    }

    private long lineScore(int[][] board, int x, int y, int dx, int dy, int
player) {
        int count = 1; // tính cả ô (x,y) đang xét như thể đã đặt quân

        int nx = x + dx, ny = y + dy;
        while (inBounds(nx, ny) && board[nx][ny] == player) {
            count++;
            nx += dx; ny += dy;
        }
        boolean openForward = inBounds(nx, ny) && board[nx][ny] == 0;

        nx = x - dx; ny = y - dy;
        while (inBounds(nx, ny) && board[nx][ny] == player) {
            count++;
            nx -= dx; ny -= dy;
        }
        boolean openBackward = inBounds(nx, ny) && board[nx][ny] == 0;

        int openEnds = (openForward ? 1 : 0) + (openBackward ? 1 : 0);
        return patternScore(count, openEnds);
    }

    private long patternScore(int count, int openEnds) {
        if (count >= 5) return 1_000_000L;
        if (count == 4) return openEnds >= 1 ? 100_000L : 50L;
        if (count == 3) return openEnds == 2 ? 5_000L : (openEnds == 1 ? 300L :
10L);
```

```
        if (count == 2) return openEnds == 2 ? 150L : (openEnds == 1 ? 25L : 2L);
        return 1L;
    }

    private boolean inBounds(int x, int y) {
        return x >= 0 && y >= 0 && x < SIZE && y < SIZE;
    }
}
```